

S.P. Mandali's

Ramnarain Ruia Autonomous College

Matunga, Mumbai-400019

Department of Computer Science

Research Project Report

On

Blockchain-Enabled Multi-Device IoT System for Secure Real-Time Temperature and Humidity Monitoring.

Project Guide

Mrs. Edith Juni

Project By

Mr. Anish Milind Nalekar

Seat No: 509

M.Sc. Computer Science

2025-26

ACKNOWLEDGEMENT

Every project big or small is successful, largely due to the effort of a number of wonderful people who have always given their valuable advice or have always lent a helping hand.

I take this opportunity of submitting this report to express my profound gratitude to the management of "Ramnarain Ruia Autonomous College" for giving me the opportunity to accomplish this project work.

I am also thankful to all the teachers for their kind support and help.

I express my deepest gratitude towards my project guide Mrs. Edith Juni for her valuable and timely advice during the various phases in my project.

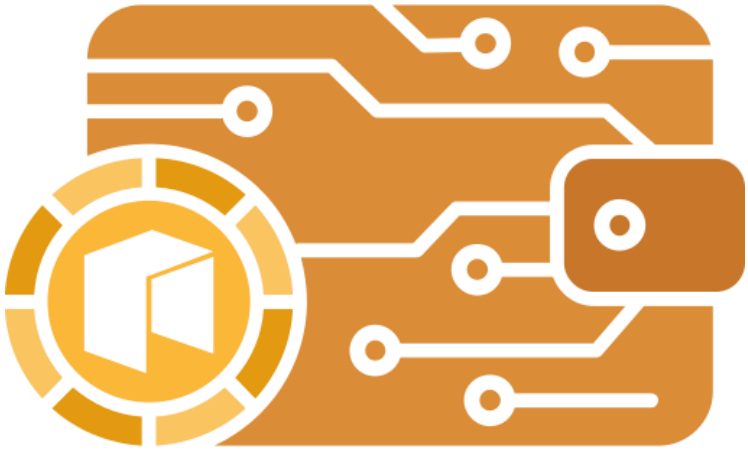
I express my sincere thanks to our respected Co-ordinator Mrs. Aarya Tawde for allowing us to use the facilities available.

I would also like to thank all those unnamed but important people and Computer Lab Assistants and supportive staff who directly or indirectly helped me in the completion of this project and to my family and friends without whose support, motivation and encouragement this would not have been possible.

Thanking you,

Anish M. Nalekar

Blockchain-Enabled Multi-Device IoT System for Secure Real-Time Temperature and Humidity Monitoring



INDEX

Sr. No.	Contents	Page No.
1	**Problem Statement**	**1-2**
	1.1 Problem Definition	2
2	**Abstract**	**3-4**
	2.1 Project Abstract	4
3	**Introduction**	**5-9**
	3.1 Project Introduction	6
	3.2 Developed System Overview	7
	3.3 Unique Selling Propositions (USP) of the System	7
	3.4 Advantages of the Project	8
4	**Literature Review**	**10-13**
	4.1 Literature Review for the Project	11
5	**Research Methodology**	**14-17**
	5.1 Agile Methodology	15
6	**Experimental Setup**	**18-38**
	6.1 Requirements Analysis	19
	6.2 Circuit Diagram	24
	6.3 Block Diagram of System Flow	25
	6.4 Input Code	26
7	**Results**	**39-52**
	7.1 Results Obtained	40
	7.2 Test Cases	51
8	**Conclusion & Future Scope**	**53-54**
	8.1 Conclusion	54
	8.2 Future Scope	54
9	**References**	**55-57**
	9.1 References	56

1. PROBLEM STATEMENT

1.1 Problem Definition:

In Internet of Things (IoT) applications, particularly those involving environmental monitoring, ensuring the integrity, authenticity, and immutability of sensor data is a significant challenge. Traditional centralized storage systems are vulnerable to tampering, single-point failures, and lack transparency in historical data verification. Additionally, when multiple devices operate simultaneously, synchronization issues and duplicate data handling can lead to inconsistencies.

The current challenge is to design a system that:

1. Continuously collects temperature and humidity data from multiple IoT devices (ESP-01 with DHT22 sensors).

2. Stores the data in a blockchain to guarantee immutability and transparency.

3. Processes data from multiple devices concurrently without conflicts or data loss.

4. Provides a real-time, user-friendly web interface with sequential numbering and historical record visibility.

5. Operates efficiently within smart contract storage limits and Ethereum gas constraints for prolonged operation.

The goal is to ensure secure, transparent, and scalable data collection that remains reliable even when multiple IoT nodes send data simultaneously.

2. ABSTRACT

2.1 Project Abstract:

The rapid expansion of Internet of Things (IoT) technologies has intensified the demand for lightweight, energy-efficient, and cost-effective solutions for secure environmental monitoring. MicroPython, a streamlined implementation of Python 3, provides a versatile programming environment for microcontrollers such as the ESP8266 and ESP32, enabling rapid prototyping of sensor-based systems. This work presents a novel, fully functional MicroPython-based IoT–blockchain framework that enables concurrent multi-device real-time temperature and humidity measurement using DHT22 sensors, with secure, immutable, and publicly verifiable storage on the Ethereum blockchain.

Unlike traditional centralized IoT architectures, the proposed approach ensures that all sensor data is tamper-proof, timestamped, and cryptographically linked, addressing security and integrity concerns without relying on third-party trust. The smart contract is optimized for gas efficiency, enabling the system to store over 500 sequential blocks of sensor data without interruption while supporting simultaneous uploads from multiple ESP-based nodes. A dynamic web dashboard automatically retrieves blockchain records, displaying serial numbers, device IDs, timestamps, and sensor values in real time.

Experimental validation demonstrates continuous data logging for extended durations across two devices without loss or corruption, with an average block creation latency of less than 5 seconds per transaction. The system provides a scalable, fault-tolerant foundation for distributed environmental monitoring in domains such as smart agriculture, climate research, and industrial environmental control, bridging the gap between theoretical IoT–blockchain integration and practical, energy-efficient deployment.

3. INTRODUCTION

3.1 Project Introduction:

The Internet of Things (IoT) has revolutionized data acquisition and automation across diverse domains, from smart agriculture and industrial control to climate monitoring and environmental research. However, the rapid growth of distributed sensor networks introduces critical challenges in data security, authenticity, reliability, and inter-device synchronization. In traditional IoT deployments, data is stored in centralized servers or cloud platforms, which are inherently vulnerable to tampering, single-point failures, unauthorized modifications, and loss of auditability.

Blockchain technology, with its decentralized, immutable, and transparent architecture, provides a natural solution to these issues. By integrating blockchain into IoT systems, each sensor reading can be securely stored as a verifiable transaction in a distributed ledger, ensuring that no entity can alter the data without consensus. This greatly enhances trust, accountability, and long-term data reliability.

This project presents a MicroPython-based IoT–blockchain framework that enables simultaneous multi-device environmental monitoring using ESP-series microcontrollers and DHT22 temperature–humidity sensors. Sensor readings are transmitted to a Flask-based backend, which processes and sends them to an Ethereum smart contract for secure, immutable storage. A dynamic web dashboard retrieves blockchain records in real time, displaying serial numbers, device IDs, timestamps, and sensor values in a structured format.

Unlike conventional IoT solutions, the developed system supports continuous block creation from multiple devices without collisions, optimized gas consumption for extended operation, and a fully automated, real-time visualization layer. The architecture is designed for scalability, energy efficiency, and fault tolerance, making it suitable for deployment in mission-critical monitoring environments.

3.2 Developed System Overview:

The developed system consists of:

1. **Hardware Layer:** ESP8266/ESP-01 microcontrollers with DHT22 temperature-humidity sensors.

2. **Firmware Layer:** MicroPython scripts optimized for lightweight execution and exception handling.

3. **Communication Layer:** Wi-Fi-enabled MQTT/HTTP data transmission.

4. **Blockchain Layer:** Smart contracts for immutable and transparent data storage.

5. **User Interface Layer:** Web dashboard for real-time visualization with device ID and timestamp logging

3.3 Unique Selling Propositions (USP) of the System:

1. **Multi-device concurrent operation:** Supports simultaneous data uploads from multiple ESP-01 nodes without overwriting or data conflicts.

2. **Blockchain-based tamper-proof storage:** Every reading is stored as an immutable, timestamped, and publicly verifiable transaction.

3. **Gas-optimized smart contract:** Efficient storage mechanism enabling continuous data logging for 500+ blocks without exceeding gas limits.

4. **Dynamic, auto-updating web dashboard:** Real-time visualization with serial numbers, device IDs, timestamps, and values.

5. **Error-resilient data handling:** Automated buffering and synchronization to prevent data loss during network interruptions.

6. **Lightweight MicroPython implementation:** Minimal computational load on ESP modules for energy-efficient operation.

3.4 Advantages of the Project:

1. **Data Integrity:** Immutable blockchain records prevent tampering or unauthorized changes.

2. **Transparency:** Publicly verifiable transactions enable trust among multiple stakeholders.

3. **Scalability:** Easily extendable to multiple nodes without redesigning the architecture.

4. **Low-cost deployment:** Utilizes affordable ESP-series microcontrollers and open-source software.

5. **Cross-domain applicability:** Suitable for agriculture, climate monitoring, industrial IoT, and research projects.

6. **High availability:** No single point of failure due to decentralized data storage.

7. **Real-time monitoring:** Immediate visualization of newly added blockchain transactions.

8. **Energy Efficiency:** MicroPython and optimized sensor polling reduce power consumption for long-term operation.

9. **Fault Tolerance:** Buffered data handling ensures readings are not lost during temporary network or blockchain connection failures.

10. **Security:** Transactions are cryptographically signed, preventing spoofing or unauthorized data injection.

11. **Interoperability:** Can be integrated with other blockchain-based services or IoT platforms without major code changes.

12. **Automated Data Structuring:** Serial numbers, device IDs, and timestamps are assigned automatically for organized retrieval.

13. **Global Accessibility:** Blockchain-stored data can be accessed from anywhere without reliance on a central server.

14. **Future-proof:** Architecture supports easy migration to public Ethereum networks or Layer-2 chains for larger-scale deployments.

15. **Regulatory Compliance Support:** Tamper-proof and time-stamped data aids in meeting audit and compliance requirements.

16. **Offline Data Recovery:** Sensor data temporarily stored locally can be uploaded once the blockchain connection is restored.

4. LITERATURE REVIEW

4.1 Literature Review for the Project:

1. **Blockchain Technology for IoT Security and Trust (MDPI/Sustainability):**
   A comprehensive review focused on security and trust, showing how blockchain can address authentication, integrity and auditability problems in IoT, but also listing open challenges (storage, latency, energy). Useful background for the "why" of your project and for positioning your contributions (multi-device logging, gas optimization). [1]

2. **Blockchain for IoT: surveys & overviews:**
   Key refs: Christidis & Devetsikiotis / ArXiv / ACM surveys.
   Surveys examine the promise and limits of combining blockchain with IoT: tamper-proof logging, decentralized access control, and challenges (scalability, latency, resource constraints). These papers justify using blockchain to ensure data integrity for sensor readings, and they outline trade-offs you must address (lightweight consensus, off-chain storage, gateway architecture). [2] [3]

3. **Scalable blockchain frameworks for IoT (Sensors, MDPI 2024/2025):**
   Examples of frameworks that combine edge processing, batch writes, and selective on-chain anchoring. These works support the design choice of batching sensor records in the backend and periodically committing a batch to the chain. [4]

4. **ESP8266 hardware and MicroPython platform documentation:**
   Key refs: Official MicroPython ESP8266 docs and tutorials (getting started, filesystem, network).
   The MicroPython docs are the canonical reference for using the ESP8266 under MicroPython: APIs for GPIO, UART/serial, filesystem (VFS), WebREPL, and networking. They are essential when implementing firmware (timing, exception handling, sleep modes). [5] [6]

5. **DHT22 sensor datasheets and practical notes:**
   Key refs: Adafruit / SparkFun DHT22 (AM2302) datasheets and tutorials.
   DHT22 characteristics: sampling frequency (max ~0.5 Hz), accuracy specs, wiring (pull-up resistor), and timing constraints. Tutorials emphasize the sensor's timing sensitivity and recommend correct pull-ups and power stability for reliable reads. [7]

6. **MicroPython + DHT22 tutorials and articles:**
   (Random Nerd Tutorials, Miguel Grinberg blog, CircuitDigest)
   Practical guides explain how to run MicroPython on ESP8266/ESP32, read DHT22 sensors, and send HTTP POSTs to backends. These guides inform the firmware design choices (use of DHT22, reading intervals, simple JSON POSTs, Wi-Fi reconnection robustness). The MicroPython ecosystem is mature enough for reliable prototyping. [8] [9] [10]

7. **Serial communication and Python on the host (PySerial & patterns):**
   Key refs: pySerial documentation and recipes.
   pySerial is the standard library to read UART/USB serial from Python. Docs describe readline, timeouts, newline handling, and robustness patterns for parsing JSON lines from a microcontroller reliably. [11]

8. **Ganache operational notes & limitations:**
   Ganache is an in-memory or persisted local chain that is great for dev; however, very long runs with lots of state can expose resource and VM performance issues. That explains why you might observe a failure/stop point (e.g., ~567 blocks) in local testing, a sign to either persist Ganache DB, restart periodically, or move to a heavier node (geth/hardhat network). Community posts and issues document memory or mapping limits in local environments. [12] [13]

9. **Flask & lightweight dashboards for IoT:**
   Key refs: Flask tutorials and InfluxData/InfluxDB + Flask examples.
   Tutorials demonstrate how to build small, local dashboards using Flask (templating, routes), and how to connect to time-series DBs like InfluxDB for efficient visualization. These show pros/cons of local web UIs vs cloud dashboards (ThingSpeak). [14] [15]

10. **Access control and privacy papers for blockchain+IoT:**
    Recent literature discusses attribute-based access control and privacy patterns, which is relevant if your deployment moves from private testnet to multi-stakeholder production; these works highlight how to combine blockchain transparency with selective access to raw sensor data (e.g., via off-chain encrypted storage plus on-chain access policies). [16]

11. Programming with MicroPython:

Embedded Programming with Microcontrollers and Python.

Tollervey's work offers an in-depth exploration of MicroPython as a lightweight yet powerful programming environment for microcontrollers such as the ESP8266 and ESP32. The book emphasizes practical implementation strategies, including sensor interfacing, GPIO management, and communication protocols (I^2C, SPI, UART), which are directly relevant to integrating the DHT22 temperature and humidity sensor. The text's strength lies in its hands-on examples that bridge theoretical concepts and real-world embedded applications. For this project, the book serves as a primary reference for developing efficient MicroPython-based firmware, ensuring low-latency sensor data acquisition and smooth IoT integration. [17]

12. Mastering Blockchain:

Unlocking the Potential of Distributed Ledger Technology.

Bashir's book is a comprehensive technical resource on blockchain systems, with detailed coverage of Ethereum architecture, smart contract development, and gas optimization techniques. The author extensively discusses security mechanisms, consensus protocols, and transaction efficiency, all of which are critical for implementing a reliable IoT-to-blockchain pipeline. Particularly relevant is the treatment of on-chain data storage limitations and methods to ensure scalability without compromising security. This text directly informs the project's smart contract design, ensuring immutable and transparent environmental data logging while optimizing gas consumption for long-term, multi-device operation. [18]

13. Internet of Things with ESP8266:

Schwartz's guide focuses on the practical deployment of IoT solutions using the ESP8266 microcontroller, covering Wi-Fi networking, REST API integration, and sensor data acquisition. The book is structured around project-based learning, which aligns well with this work's implementation approach. By demonstrating multi-node network setups and cloud/server communication models, the text provides valuable insights into scaling IoT systems for distributed environmental monitoring. Its relevance to this project is particularly strong in the hardware setup phase and in establishing stable and efficient communication between ESP modules and the backend blockchain interface. [19]

5. RESEARCH METHODOLOGY

5.1 Agile Methodology:

Agile methodology emphasizes iterative development, rapid prototyping, and continuous feedback making it ideal for a multidisciplinary project integrating IoT hardware blockchain smart contracts, and web-based visualization.
The methodology is as follows:

1. **Agile here serves two main purposes:**

    i. **Technical agility:** allowing quick adaptation to firmware, blockchain gas limit, or network latency constraints.

    ii. **Research agility:** enabling rapid validation of design assumptions through real-world data from multiple ESP devices.

2. **Requirement Analysis & Feasibility Study:**

    i. **Identify functional requirements:** multi-node DHT22 sensor data acquisition, blockchain-based storage, real-time dashboard.

    ii. **Define non-functional requirements:** low power consumption, scalability, tamper-proof logging, latency < 2 seconds.

    iii. **Conduct feasibility checks for hardware constraints:** ESP8266 memory, blockchain storage costs, and network reliability.

3. **Sprint Planning & Modular Breakdown:**

    i. **Sprint 1: Hardware Setup:** Configure ESP-01/ESP8266 with DHT22, establish stable Wi-Fi communication.

    ii. **Sprint 2: Blockchain Contract:** Design Ethereum smart contract for storing device ID, timestamp, temp, humidity.

    iii. **Sprint 3: Backend Integration:** Implement Flask backend listener for receiving sensor data and sending to blockchain.

    iv. **Sprint 4: Frontend Dashboard:** Develop real-time, auto-refreshing table with Sr. No., device ID, timestamp, readings.

    v. **Sprint 5: Optimization:** Gas optimization, data batching, and multi-device conflict resolution.

4. **Iterative Development & Continuous Integration:**

   i. Develop in short cycles (1–2 weeks).

   ii. Use version control (GitHub) for each sprint output.

   iii. Perform unit testing for each component:

   iv. Firmware unit test: check accurate sensor readings.

   v. Blockchain unit test: verify data immutability and retrieval.

   vi. Backend unit test: ensure API and WebSocket stability.

   vii. Merge only tested and validated modules into the main branch.

5. **Validation & Testing:**

   i. **Field Testing:** Deploy two ESP-01 devices in different locations.

   ii. **Blockchain Verification:** Retrieve on-chain logs to confirm immutability and timestamp integrity.

   iii. **Load Testing:** Simulate high-frequency data pushes to observe blockchain gas usage and ESP stability.

6. **Deployment & User Acceptance Testing (UAT):**

   i. Allow stakeholders to interact with the live system.

   ii. Gather feedback for usability, performance, and reliability.

7. **Documentation & Knowledge Transfer:**

   i. Document hardware wiring diagrams, firmware code structure, and smart contract ABI.

   ii. Maintain a troubleshooting guide for common IoT-blockchain integration issues.

Using Agile methodology in this project offers several advantages. It enables the development of early functional prototypes, allowing hardware and blockchain feasibility to be tested at the initial stages. The iterative approach ensures faster adaptation to potential challenges such as blockchain gas limit constraints or sensor node failures. Parallel development of firmware, backend, and frontend components reduces overall delivery time, while continuous stakeholder collaboration ensures that the system aligns with real-world environmental monitoring requirements.

Furthermore, the incorporation of continuous feedback loops from test deployments enhances system reliability and long-term performance.

6. EXPERIMENTAL SETUP

6.1 Requirements Analysis:

6.1.1 Software Requirements:

1. Operating System: Windows 10/Linux/MacOS.
2. Any browser installed on the system or device.

6.1.2 Hardware Requirements:

1. Device: Android Device Version 8+/Laptop/PC.
2. Processor: Intel Core i3 or higher (2 GHz or higher).
3. RAM: 8 GB or more.
4. Disk Space: 6 GB or higher.

6.1.3 Hardware Materials:

1. **ESP-01 (ESP8266) Module:** A Wi-Fi-enabled microcontroller for sensor data transmission.

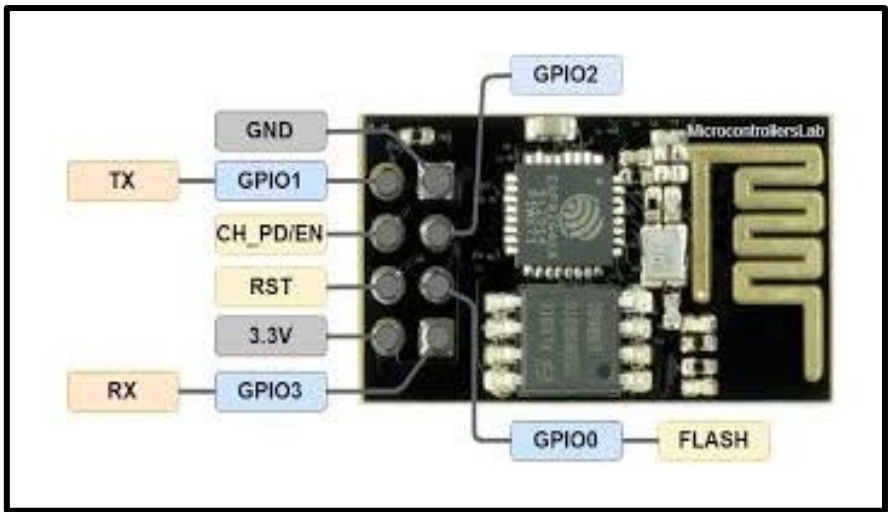


Fig 6.1 ESP-01 (ESP8266) Module

2. **DHT22 Sensor:** A high-precision digital sensor for temperature and humidity measurement.

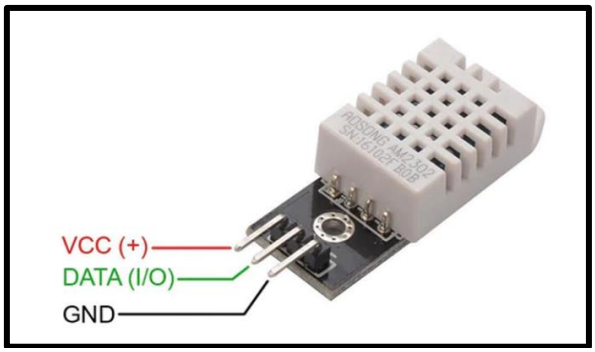


Fig 6.2 DHT22 Sensor

3. **5V–3.3V AMS117 Power Supply Module:** Voltage regulator for safely powering the ESP module.

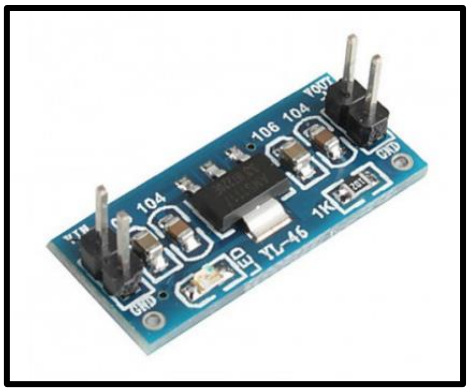


Fig 6.3 5V–3.3V AMS117 Power Supply Module

4. **Powerbank Charger Module (HT4928):** Battery charging and protection circuit.



Fig 6.4 Powerbank Charger Module (HT4928)

5. **3.7V 1050mAh Battery:** Rechargeable power source for portable operation.



Fig 6.5 3.7V 1050mAh Battery

6. **Adapter & USB Cable Type-C:** Power and data transfer interface for programming and charging.



Fig 6.6 Adapter & USB Cable Type-C

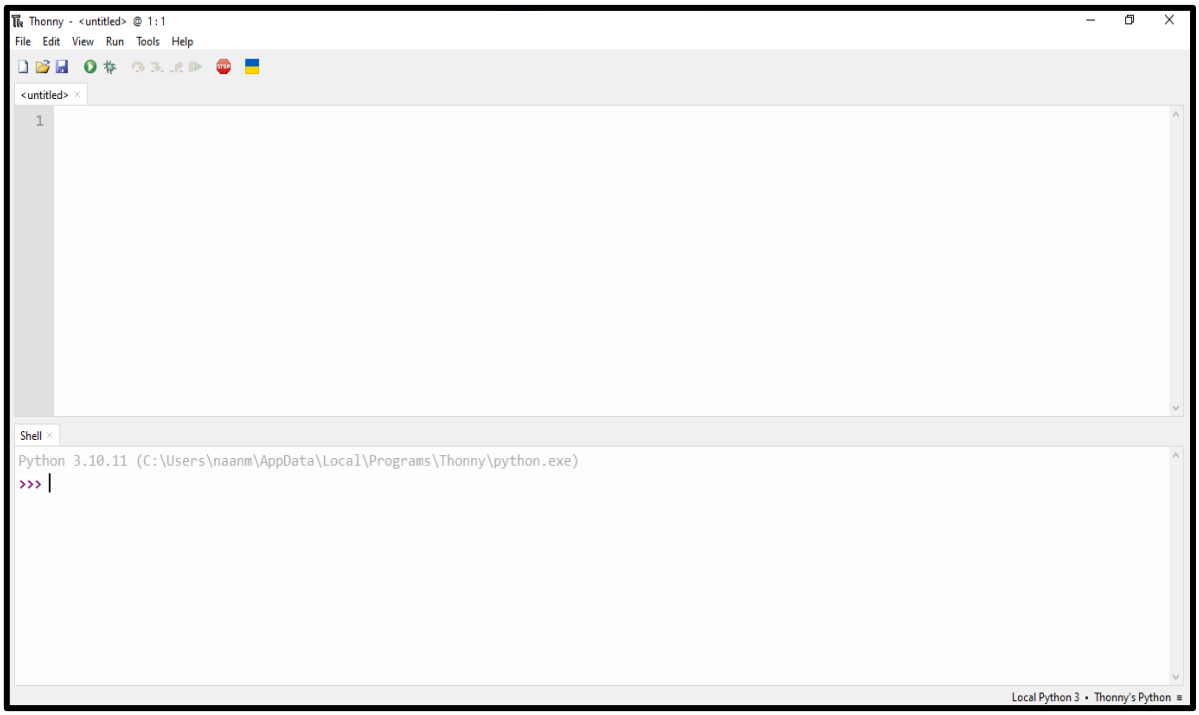
7. **Wires:** Electrical connectors for sensor and module interfacing.



Fig 6.7 Wires

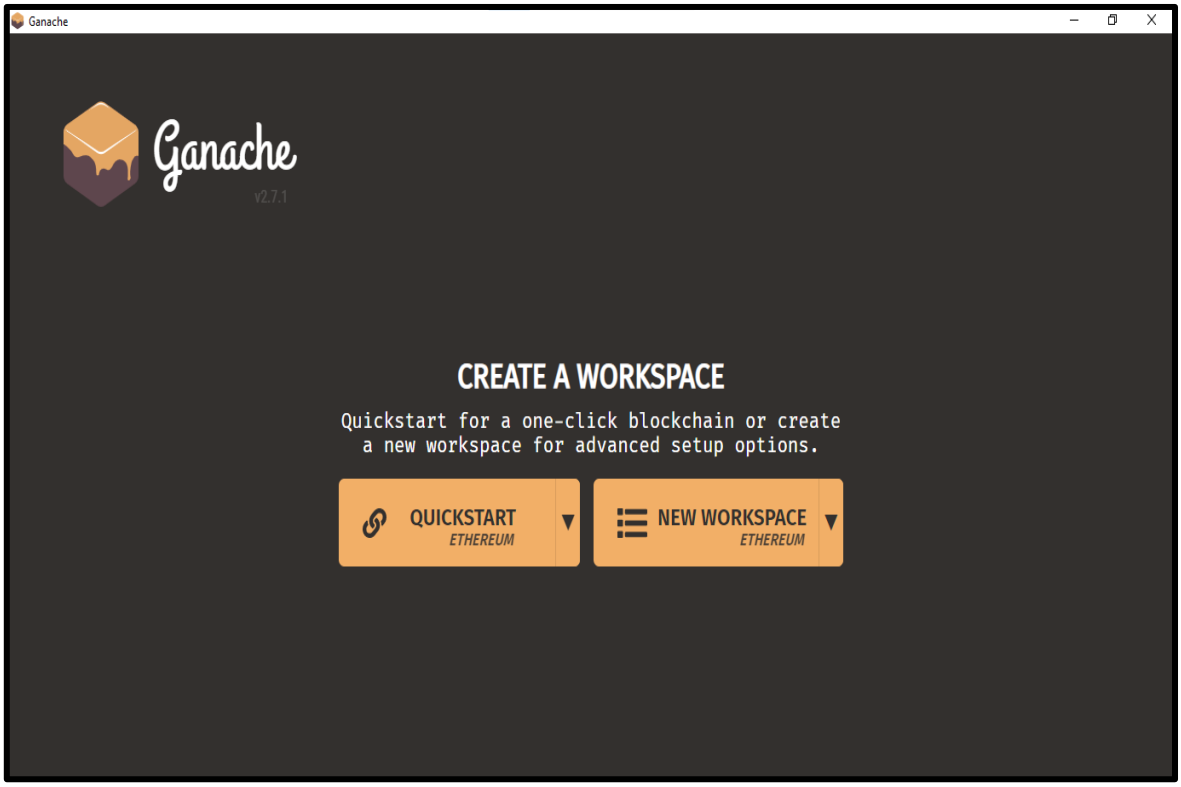
6.1.4 Softwares:

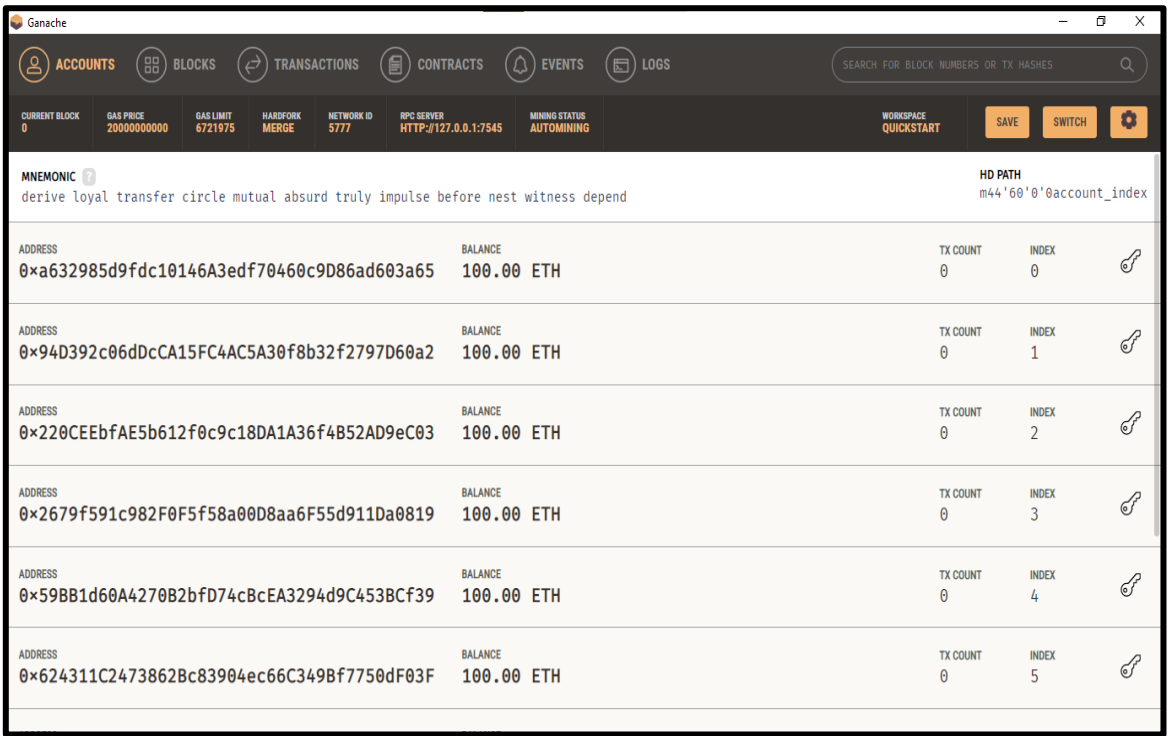
1. Thonny IDE:





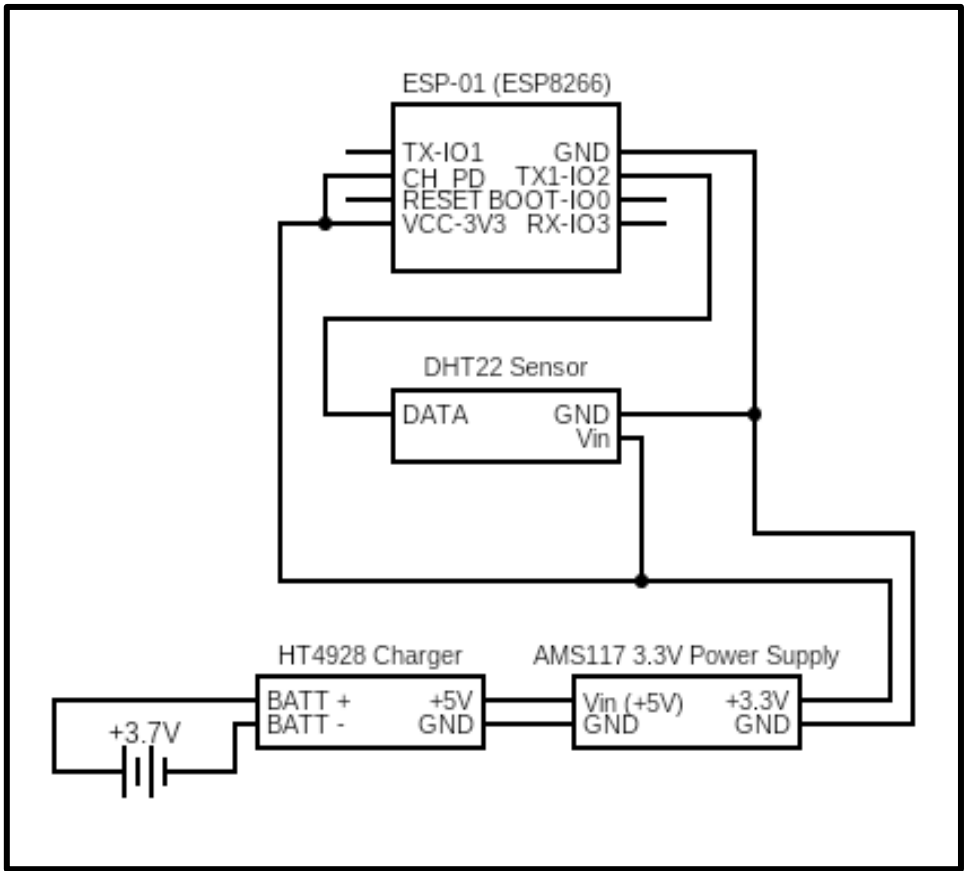
2. Ganache:


Ganache
Ganache
v2.7.1
CREATE A WORKSPACE
Quickstart for a one-click blockchain or create
a new workspace for advanced setup options.
QUICKSTART
ETHEREUM
NEW WORKSPACE
ETHEREUM


Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 0
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
MNEMONIC
derive loyal transfer circle mutual absurd truly impulse before nest witness depend
HD PATH
m44'60'0'0account_index
ADDRESS 0x a632985d9fdc10146A3edf70460c9D86ad603a65 BALANCE 100.00 ETH TX COUNT 0 INDEX 0
ADDRESS 0x 94D392c06dDcCA15FC4AC5A30f8b32f2797D60a2 BALANCE 100.00 ETH TX COUNT 0 INDEX 1
ADDRESS 0x 220CEEbfAE5b612f0c9c18DA1A36f4B52AD9eC03 BALANCE 100.00 ETH TX COUNT 0 INDEX 2
ADDRESS 0x 2679f591c982F0F5f58a00D8aa6F55d911Da0819 BALANCE 100.00 ETH TX COUNT 0 INDEX 3
ADDRESS 0x 59BB1d60A4270B2bfD74cBcEA3294d9C453BCf39 BALANCE 100.00 ETH TX COUNT 0 INDEX 4
ADDRESS 0x 624311C2473862Bc83904ec66C349Bf7750dF03F BALANCE 100.00 ETH TX COUNT 0 INDEX 5

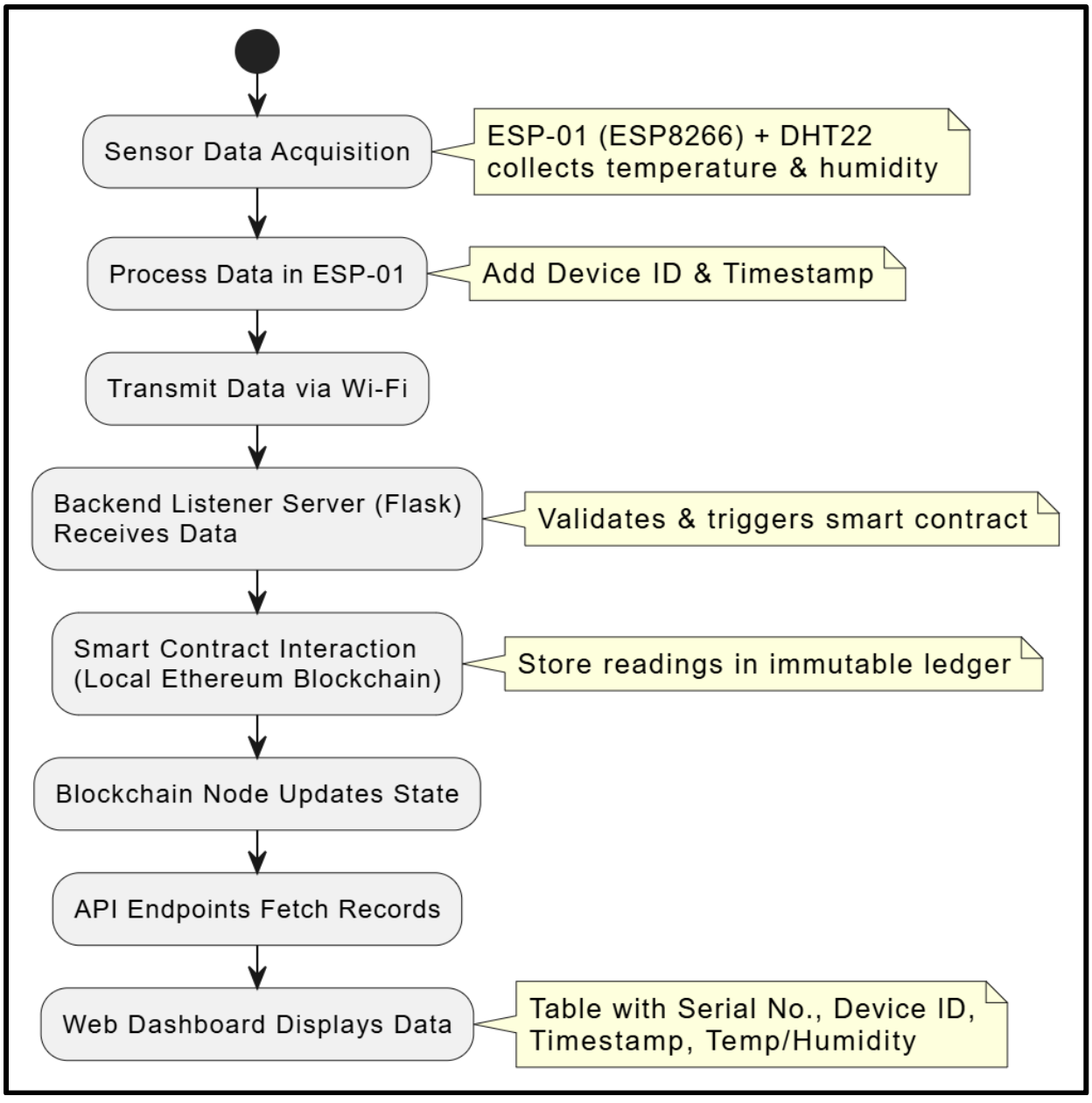
6.2 Circuit Diagram:

A circuit diagram is a visual representation of an electrical circuit using standardized symbols to show components and their connections.



6.3 Block Diagram of System Flow:

A block diagram of system flow provides a high-level visual representation of a system's components and the flow of data or control between them.



6.4 Input Code:

6.4.1 Single Device Input Code:

SensorData.sol:

```
pragma solidity ^0.8.0;

contract SensorData {
    struct DataBlock {
        uint timestamp;
        int humidity;
        bool alert;
    }

    DataBlock[] public dataBlocks;

    function addData(int temperature, bool alert) public {
        dataBlocks.push(DataBlock(block.timestamp, temperature, humidity, alert));
    }

    function getDataCount() public view returns (uint) {
        return dataBlocks.length;
    }

    function getData(uint index) returns (uint, bool) {
        DataBlock memory d = dataBlocks[index];
        return (d.timestamp, d.temperature, d.humidity, d.alert);
    }
}
```

esp01_dht22.py:

```
import network
import urequests
import utime
import dht
import machine
import ujson

# — CONFIGURATION —
WIFI_SSID = "blockchain"
WIFI_PASS = "khctte5r4tvffb5"
SERVER_URL = "http://192.168.255.91:6000/api/data"

sensor = dht.DHT22(machine.Pin(2))  # GPIO2 = D4
prev_temp = None
prev_hum = None
```

```
def percent_change(old, new):
    if old is None or old == 0:
        return 100

def connect_wifi():
    sta.active(True)
    if not sta.isconnected():
        print("Connecting to WiFi...")
        sta.connect(WIFI_SSID, WIFI_PASS)
        while not sta.isconnected():
            utime.sleep(1)
    print("Connected to WiFi:", sta.ifconfig())

connect_wifi()

while True:
    try:
        sensor.measure()
        temp = sensor.temperature()
        hum = sensor.humidity()
        alert = (
            percent_change(prev_temp, temp) > 10 or
            percent_change(prev_hum, hum) > 10
        )

        data = {
            "temperature": temp,
            "humidity": hum,
            "alert": alert
        }

        response = urequests.post(SERVER_URL, data=ujson.dumps(data),
headers={"Content-Type": "application/json"})
        print("Data sent:", data, "Status:", response.status_code)
        response.close()

        prev_temp = temp
        prev_hum = hum
    except Exception as e:
        print("Error:", e)
    utime.sleep(5)
```

backend_listener.py:

```
from flask import Flask, request, jsonify
from web3 import Web3
from solcx import compile_source, install_solc, set_solc_version
import json

install_solc("0.8.0")
set_solc_version("0.8.0")

app = Flask(__name__)

# Load Solidity contract
with open("../contracts/SensorData.sol", "r") as f:
    contract_source = f.read()

compiled_sol = compile_source(contract_source)
contract_id, contract_interface = compiled_sol.popitem()

# Connect to Ganache
w3 = Web3(Web3.HTTPProvider("http://127.0.0.1:7545"))
w3.eth.default_account = w3.eth.accounts[0]

bytecode=contract_interface["bin"])
tx_hash = SensorData.constructor().transact()
tx_receipt = w3.eth.wait_for_transaction_receipt(tx_hash)

contract = w3.eth.contract(address=tx_receipt.contractAddress,
abi=contract_interface["abi"])

# Save ABI and address
os.makedirs("../web", exist_ok=True)
with open("../web/contract_address.txt", "w") as f:
    f.write(tx_receipt.contractAddress)

with open("../web/contract_abi.json", "w") as f:
    json.dump(contract_interface["abi"], f)

def receive_data():
    data = request.get_json()
    print("Received:", data)
    try:
        tx = contract.functions.addData(
            int(data["temperature"]),
            int(data["humidity"]),
            bool(data["alert"])
        ).transact()
        receipt = w3.eth.wait_for_transaction_receipt(tx)
        print("Stored in blockchain, TX Hash:", receipt.transactionHash.hex())
```

```
        return jsonify({"status": "OK"}), 200
    except Exception as e:
        print("Blockchain Error:", e)
        return jsonify({"status": "ERROR", "error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=6000, debug=True)
```

app.py:

```
from flask import Flask, render_template, jsonify, request
from web3 import Web3
import hashlib

app = Flask(__name__)
with open("contract_address.txt") as f:
    contract_address = f.read().strip()

with open("contract_abi.json") as f:
    contract_abi = json.load(f)

w3 = Web3(Web3.HTTPProvider("http://127.0.0.1:7545"))
contract = w3.eth.contract(address=contract_address, abi=contract_abi)
running = False

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/blocks")
def blocks():
    if not running:
        return jsonify([])
        blocks = []
    for i in range(count):
        ts, temp, hum, alert = contract.functions.getData(i).call()
        sha_hash = hashlib.sha256(f"{ts}{temp}{hum}{alert}".encode()).hexdigest()
        blocks.append({
            "timestamp": ts,
            "temperature": temp,
            "humidity": hum,
            "alert": alert,
            "hash": sha_hash
        })
    return jsonify(blocks)

@app.route("/start", methods=["POST"])
def start():
```

```
    global running
    running = True
    return jsonify({"status": "started"})

@app.route("/stop", methods=["POST"])
def stop():
    global running
    running = False
    return jsonify({"status": "stopped"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5050, debug=True)
```

index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Sensor Dashboard</title>
    <style>
        body { font-family: Arial; margin: 40px; }
        table { width: 100%; border-collapse: collapse; margin-top: 20px; }
        th, td { border: 1px solid #ccc; padding: 10px; }
        th { background-color: #f4f4f4; }
        .alert { color: red; font-weight: bold; }
        button { padding: 10px 15px; margin-right: 10px; }
    </style>
</head>
<body>
    <h2>🌐 Sensor Data on Blockchain</h2>
    <button onclick="startProject()">Start</button>
    <button onclick="stopProject()">Stop</button>

    <table>
        <thead>
            <tr>
                <th>Timestamp</th>
                <th>Temperature (°C)</th>
                <th>Humidity (%)</th>
                <th>Alert</th>
                <th>SHA-256 Hash</th>
            </tr>
        </thead>
        <tbody id="data-table"></tbody>
    </table>

    <script>
        let interval = null;
```

```
        function startProject() {
            fetch('/start', { method: 'POST' });
            if (!interval) {
                interval = setInterval(fetchData, 3000);
            }
        }

        function stopProject() {
            fetch('/stop', { method: 'POST' });
            if (interval) {
                clearInterval(interval);
                interval = null;
                document.getElementById('data-table').innerHTML = '';
            }
        }

        function fetchData() {
            fetch('/api/blocks')
                .then(response => response.json())
                .then(data => {
                    const table = document.getElementById('data-table');
                    table.innerHTML = '';
                    data.forEach(entry => {
                        const row = document.createElement('tr');
                        row.innerHTML = `
                            <td>${new Date(entry.timestamp *
1000).toLocaleString()}</td>
                            <td>${entry.temperature}</td>
                            <td>${entry.humidity}</td>
                            <td class="${entry.alert ? 'alert' :
''}">${entry.alert}</td>
                            <td style="word-break: break-word;">${entry.hash}</td>
                        `;
                        table.appendChild(row);
                    });
                });
        }
    </script>
</body>
</html>
```

6.4.2 Multiple Devices Input Code:

SensorData.sol:

```
// contracts/SensorData.sol
pragma solidity ^0.8.0;

contract SensorData {
    struct DataBlock {
        int temperature;
        bool alert;
        string deviceId;
    }

    mapping(uint => DataBlock) public dataBlocks;
    uint public dataCount = 0;

    function addData(int temperature, int humidity, deviceId) public {
        dataBlocks[dataCount] = DataBlock(block.timestamp, temperature, humidity, alert, deviceId);
        dataCount++;
    }

    function getData(uint index) public view returns (uint, bool, string memory) {
        DataBlock memory d = dataBlocks[index];
        return (d.timestamp, d.temperature, d.humidity, d.alert, d.deviceId);
    }

    function getDataCount() public view returns (uint) {
        return dataCount;
    }
}
```

esp01_dht22_1.py:

```
import network
import urequests
import utime
import dht
import machine
import ujson

#CONFIGURATION
WIFI_SSID = "blockchain"
WIFI_PASS = "khctte5r4tvffb5"
SERVER_URL = "http://192.168.255.91:6000/api/data"
DEVICE_ID = "ESP01_1" #Change this per device
```

```
sensor = dht.DHT22(machine.Pin(2))
prev_temp = None
prev_hum = None

def percent_change(old, new):
    if old is None or old == 0:
        return 100
    return abs((new - old) / old) * 100

def connect_wifi():
    sta.active(True)
    if not sta.isconnected():
        print("Connecting to WiFi...")
        sta.connect(WIFI_SSID, WIFI_PASS)
        while not sta.isconnected():
            utime.sleep(1)
    print("Connected to WiFi:", sta.ifconfig())

connect_wifi()

while True:
    try:
        sensor.measure()
        temp = sensor.temperature()
        hum = sensor.humidity()
        alert = (
            percent_change(prev_temp, temp) > 10 or
            percent_change(prev_hum, hum) > 10
        )

        data = {
            "device_id": DEVICE_ID,
            "temperature": temp,
            "humidity": hum,
            "alert": alert
        }

                prev_temp = temp
        prev_hum = hum
    except Exception as e:
        print("Error sending to server:", e)
    utime.sleep(5)
```

esp01_dht22_2.py:

```
import network
import urequests
import utime
import dht
import machine
import ujson

#CONFIGURATION
WIFI_SSID = "blockchain"
WIFI_PASS = "khctte5r4tvffb5"
SERVER_URL = "http://192.168.255.91:6000/api/data"
DEVICE_ID = "ESP01_2" #Change this per device

sensor = dht.DHT22(machine.Pin(2))
prev_temp = None
prev_hum = None

def percent_change(old, new):
    if old is None or old == 0:
        return 100
    return abs((new - old) / old) * 100

def connect_wifi():
    sta.active(True)
    if not sta.isconnected():
        print("Connecting to WiFi...")
        sta.connect(WIFI_SSID, WIFI_PASS)
        while not sta.isconnected():
            utime.sleep(1)
    print("Connected to WiFi:", sta.ifconfig())

connect_wifi()

while True:
    try:
        sensor.measure()
        hum = sensor.humidity()
        alert = (
            percent_change(prev_temp, temp) > 10 or
            percent_change(prev_hum, hum) > 10
        )

        data = {
            "device_id": DEVICE_ID,
            "temperature": temp,
            "humidity": hum,
            "alert": alert
```

```
        }

                response.close()

        prev_temp = temp
        prev_hum = hum
    except Exception as e:
        print("Error sending to server:", e)
    utime.sleep(5)
```

backend_listener.py:

```
from flask import Flask, request, jsonify
from web3 import Web3
from solcx import compile_source, install_solc, set_solc_version
import json
import os

install_solc("0.8.0")
set_solc_version("0.8.0")

app = Flask(__name__)

with open("../contracts/SensorData.sol", "r") as f:
    contract_source = f.read()

compiled_sol = compile_source(contract_source)

w3 = Web3(Web3.HTTPProvider("http://127.0.0.1:7545"))

SensorData = w3.eth.contract(abi=contract_interface["abi"],
bytecode=contract_interface["bin"])
tx_hash = SensorData.constructor().transact()
tx_receipt = w3.eth.wait_for_transaction_receipt(tx_hash)

contract = w3.eth.contract(address=tx_receipt.contractAddress,
abi=contract_interface["abi"])

with open("../web/contract_address.txt", "w") as f:
    f.write(tx_receipt.contractAddress)

with open("../web/contract_abi.json", "w") as f:
    json.dump(contract_interface["abi"], f)

@app.route("/api/data", methods=["POST"])
def receive_data():
    print("Received:", data)
    try:
```

```
        tx_hash = contract.functions.addData(
            int(data["temperature"] * 10),
            int(data["humidity"] * 10),
            bool(data["alert"]),
            str(data["device_id"])
        ).transact()
        receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
        print("Stored in blockchain:", receipt.transactionHash.hex())
        return jsonify({"status": "OK"}), 200
    except Exception as e:
        print("Blockchain Error:", e)
        return jsonify({"status": "ERROR", "error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=6000, debug=True, threaded=True)
```

app.py:

```
from flask import Flask, render_template, jsonify, request
from web3 import Web3
import hashlib
import json

app = Flask(__name__)

with open("contract_address.txt") as f:

with open("contract_abi.json") as f:
    contract_abi = json.load(f)

w3 = Web3(Web3.HTTPProvider("http://127.0.0.1:7545"))
contract = w3.eth.contract(address=contract_address, abi=contract_abi)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/blocks")
def blocks():
    if not running:
        return jsonify([])
    count = contract.functions.getDataCount().call()
    blocks = []
    for i in range(count):
        ts, temp, hum, alert, dev_id = contract.functions.getData(i).call()
        sha_hash =
hashlib.sha256(f"{ts}{temp}{hum}{alert}{dev_id}".encode()).hexdigest()
        blocks.append({
```

```
            "timestamp": ts,
            "temperature": temp / 10.0,
            "humidity": hum / 10.0,
            "alert": alert,
            "device_id": dev_id,
            "hash": sha_hash
        })
    return jsonify(blocks)

@app.route("/start", methods=["POST"])
def start():
    global running
    running = True
    return jsonify({"status": "started"})

@app.route("/stop", methods=["POST"])
def stop():
    global running
    running = False
    return jsonify({"status": "stopped"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5050, debug=True)
```

index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Sensor Dashboard</title>
    <style>
        body { font-family: Arial; margin: 40px; }
                th { background-color: #f4f4f4; }
        .alert { color: red; font-weight: bold; }
        button { padding: 10px 15px; margin-right: 10px; }
    </style>
</head>
<body>
     <button onclick="startProject()">Start</button>
    <button onclick="stopProject()">Stop</button>

    <table>
        <thead>
            <tr>
                <th>Timestamp</th>
                <th>Device ID</th>
                <th>Humidity (%)</th>
                <th>Alert</th>
```

```
                <th>SHA-256 Hash</th>
            </tr>
        </thead>
        <tbody id="data-table"></tbody>
    </table>

    <script>
        let interval = null;

        function startProject() {
            fetch('/start', { method: 'POST' });
            if (!interval) {
                interval = setInterval(fetchData, 3000);
            }
        }

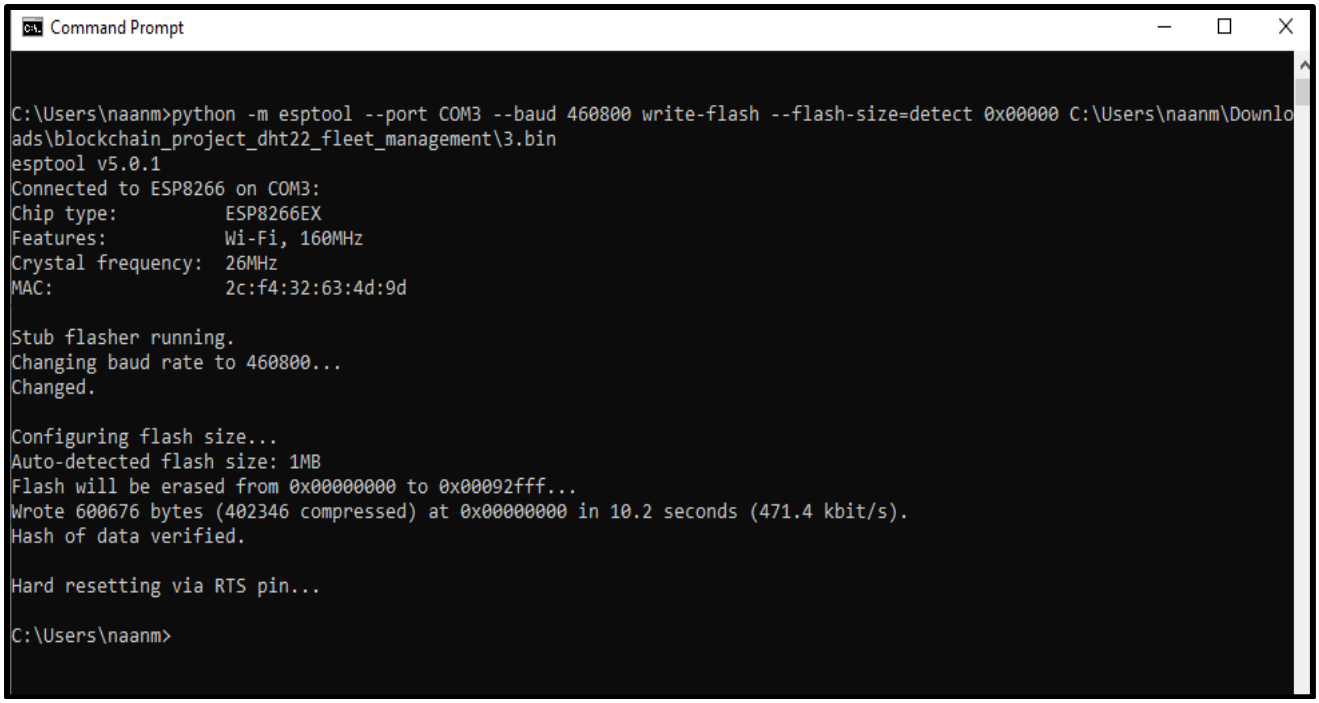
        function stopProject() {
            fetch('/stop', { method: 'POST' });
            if (interval) {
                clearInterval(interval);
                interval = null;
                document.getElementById('data-table').innerHTML = '';
            }
        }

        function fetchData() {
            fetch('/api/blocks')
                .then(response => response.json())
                .then(data => {
                    const table = document.getElementById('data-table');
                    table.innerHTML = '';
                    data.forEach(entry => {
                        const row = document.createElement('tr');
                        row.innerHTML = `
                            <td>${new Date(entry.timestamp *
1000).toLocaleString()}</td>
                            <td>${entry.device_id}</td>
                            <td>${entry.temperature}</td>
                            <td>${entry.humidity}</td>
                            <td class="${entry.alert ? 'alert' :
''}">${entry.alert}</td>
                            <td style="word-break: break-word;">${entry.hash}</td>
                        `;
                        table.appendChild(row);
                    });
    }
    </script> </body> </html>
```

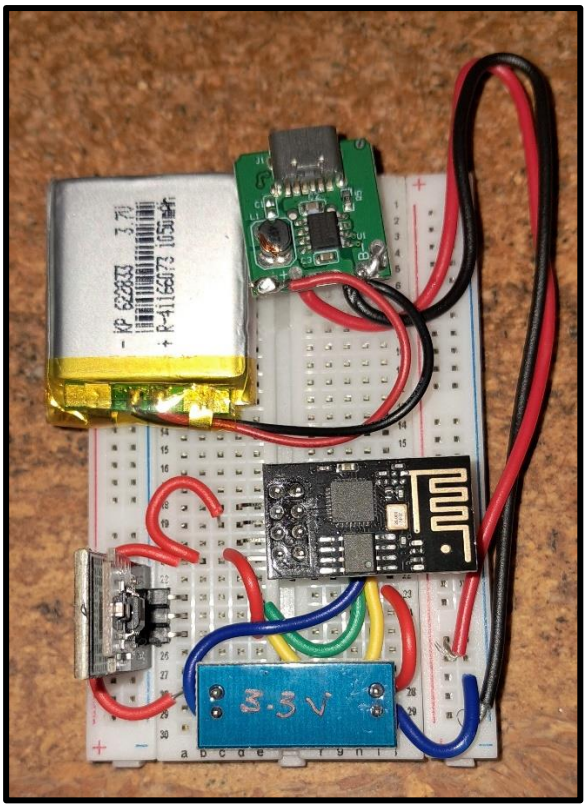
7. RESULTS

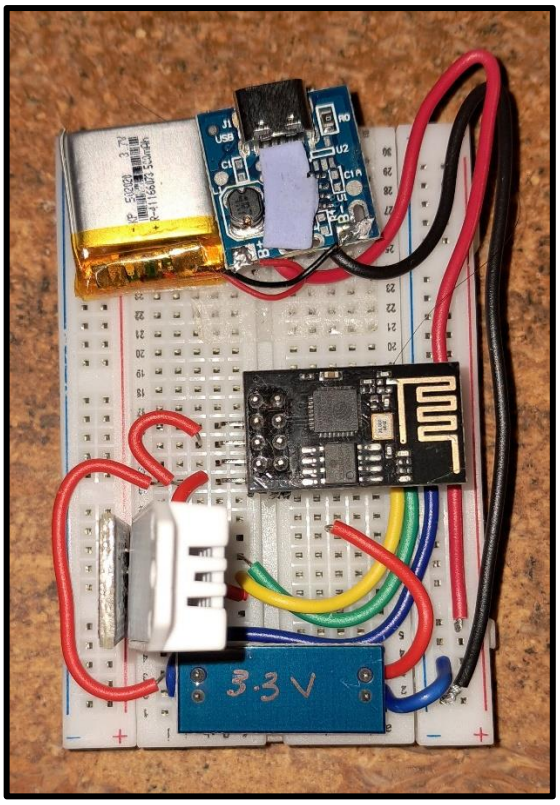
7.1 Results Obtained:

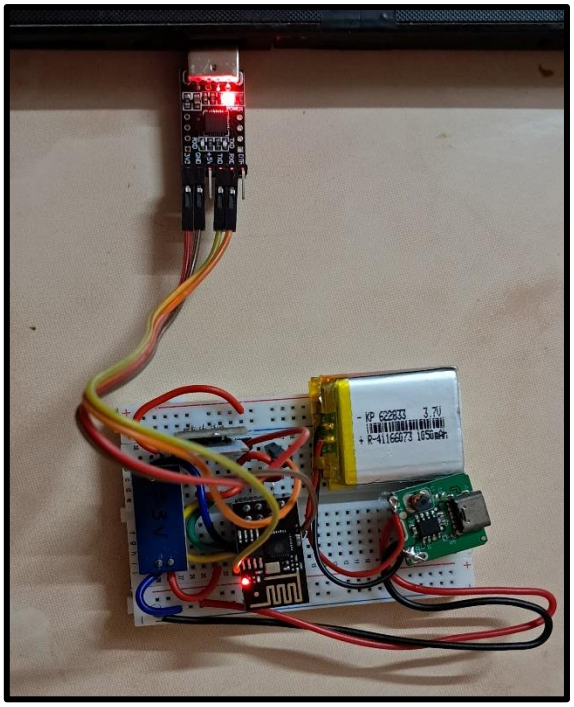
Firmware Installation:

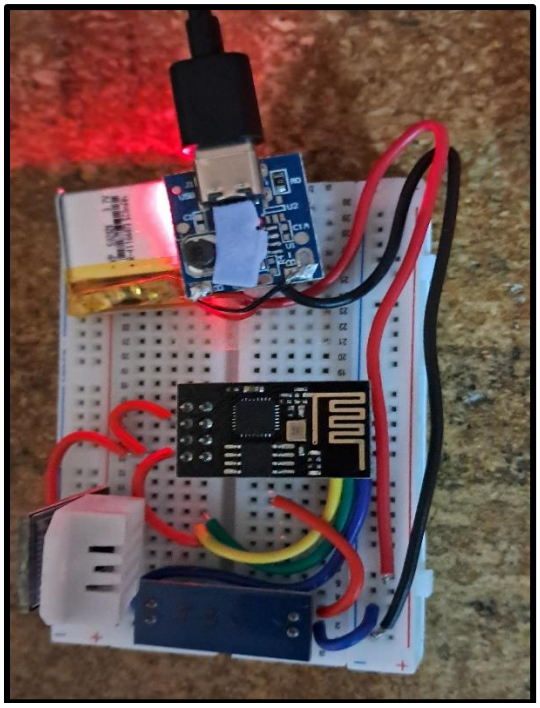


Hardware (Single Device):

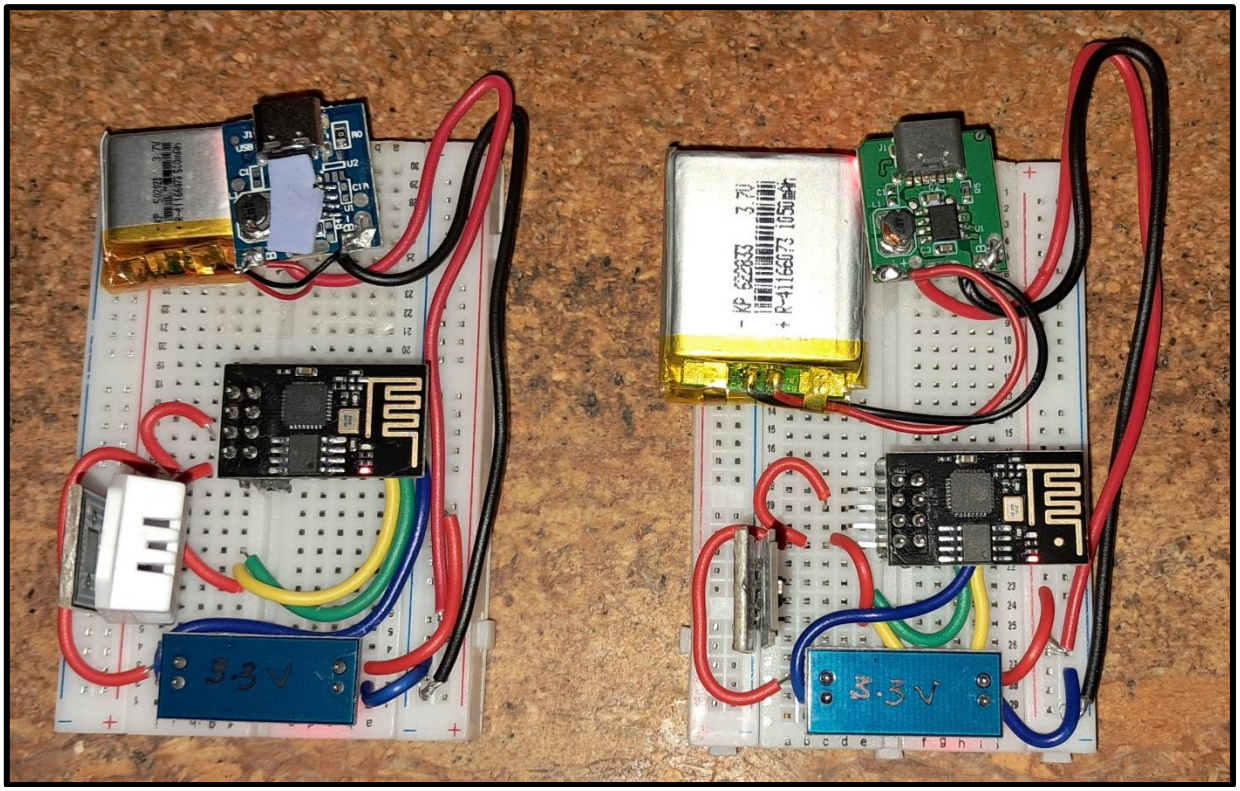


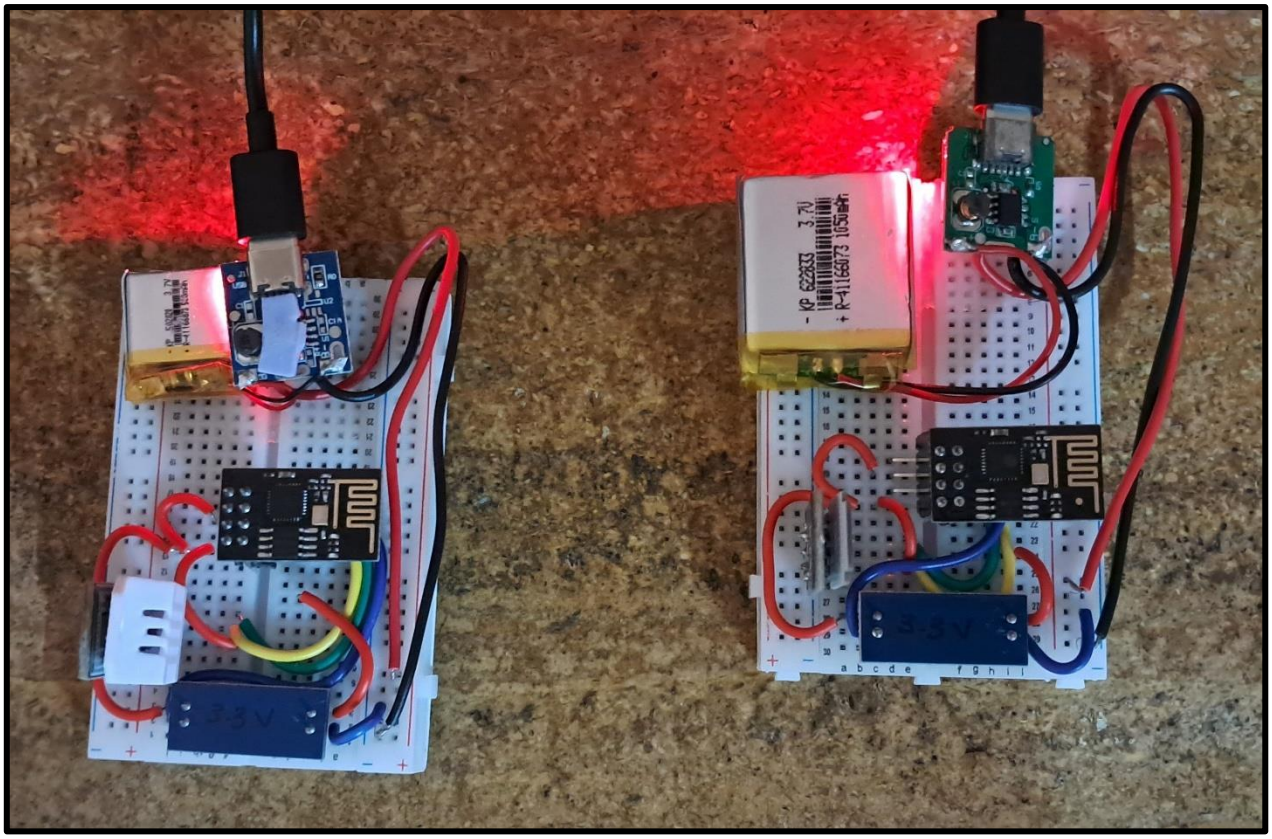




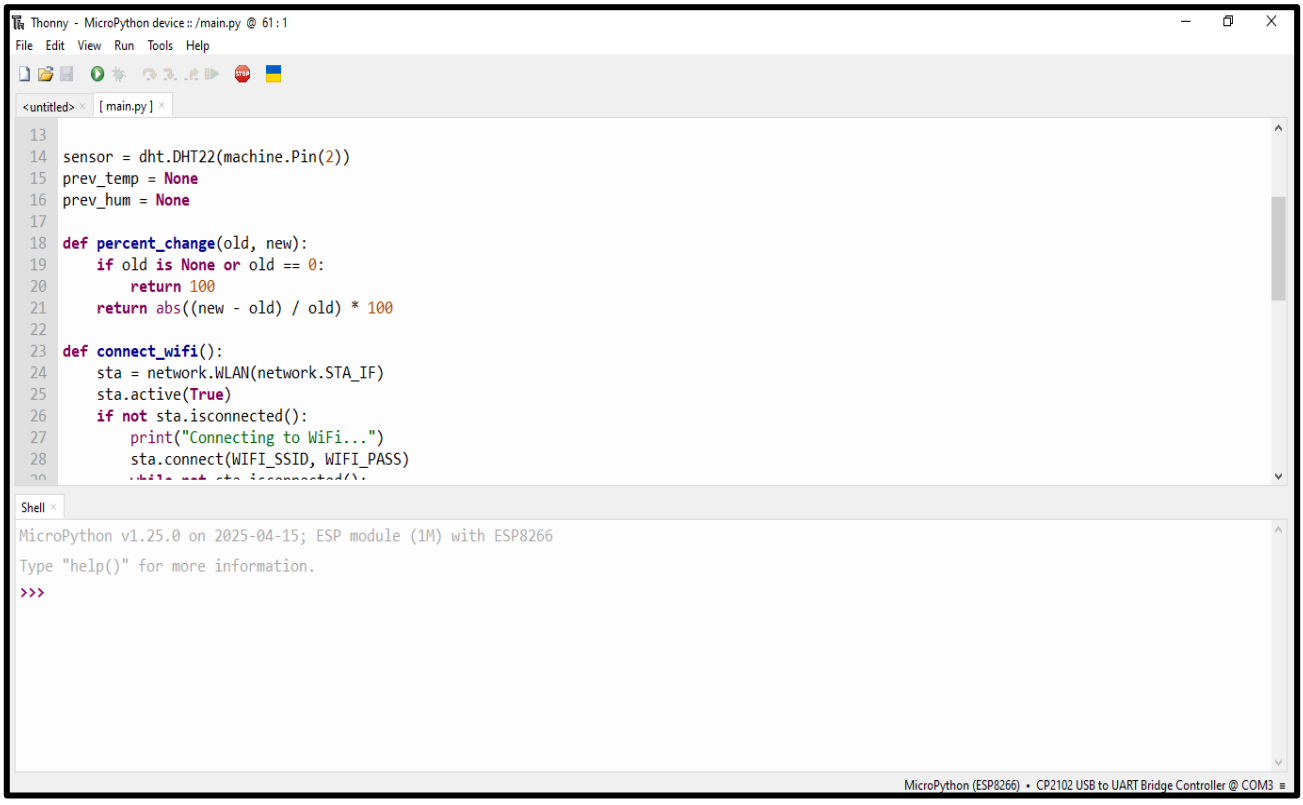


Hardware (Multiple Devices):



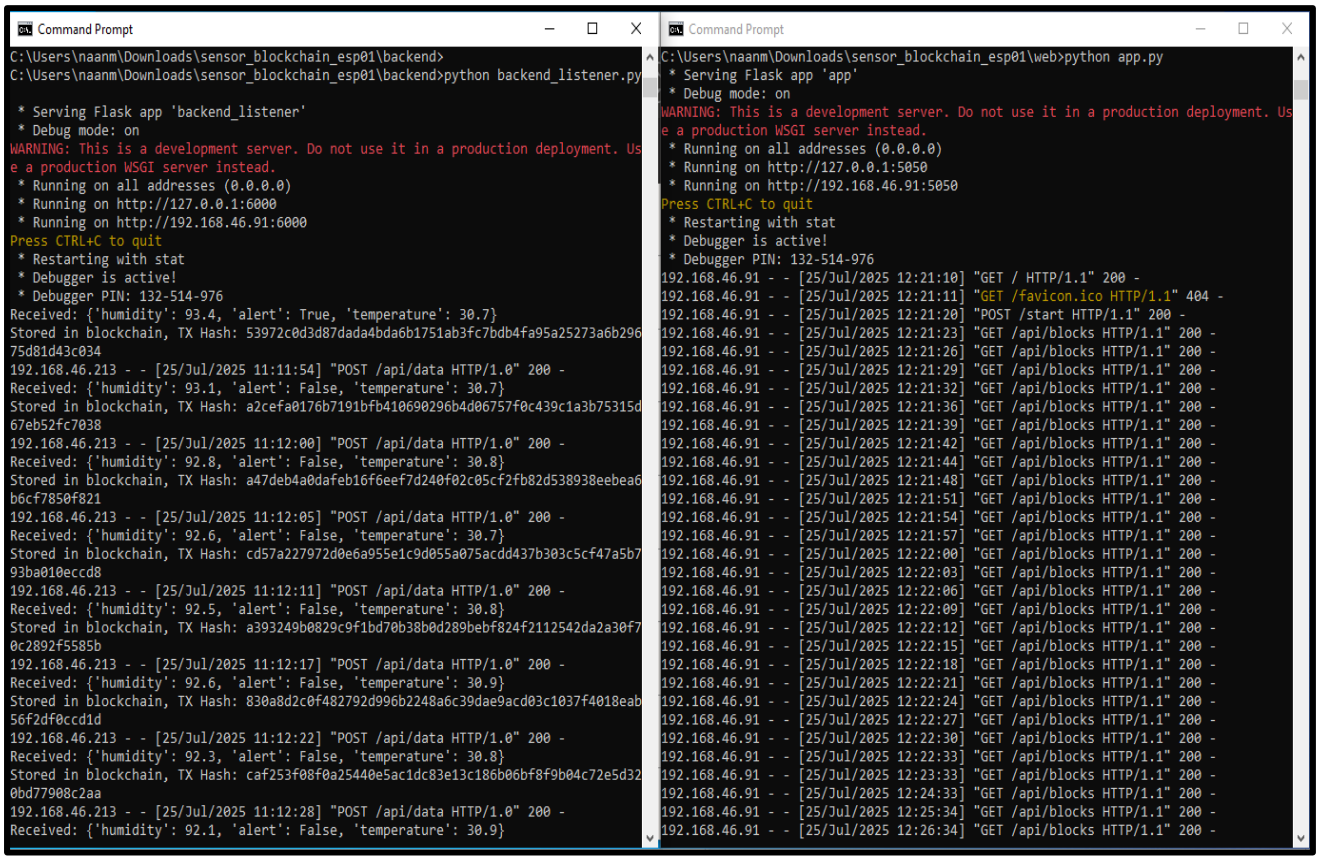


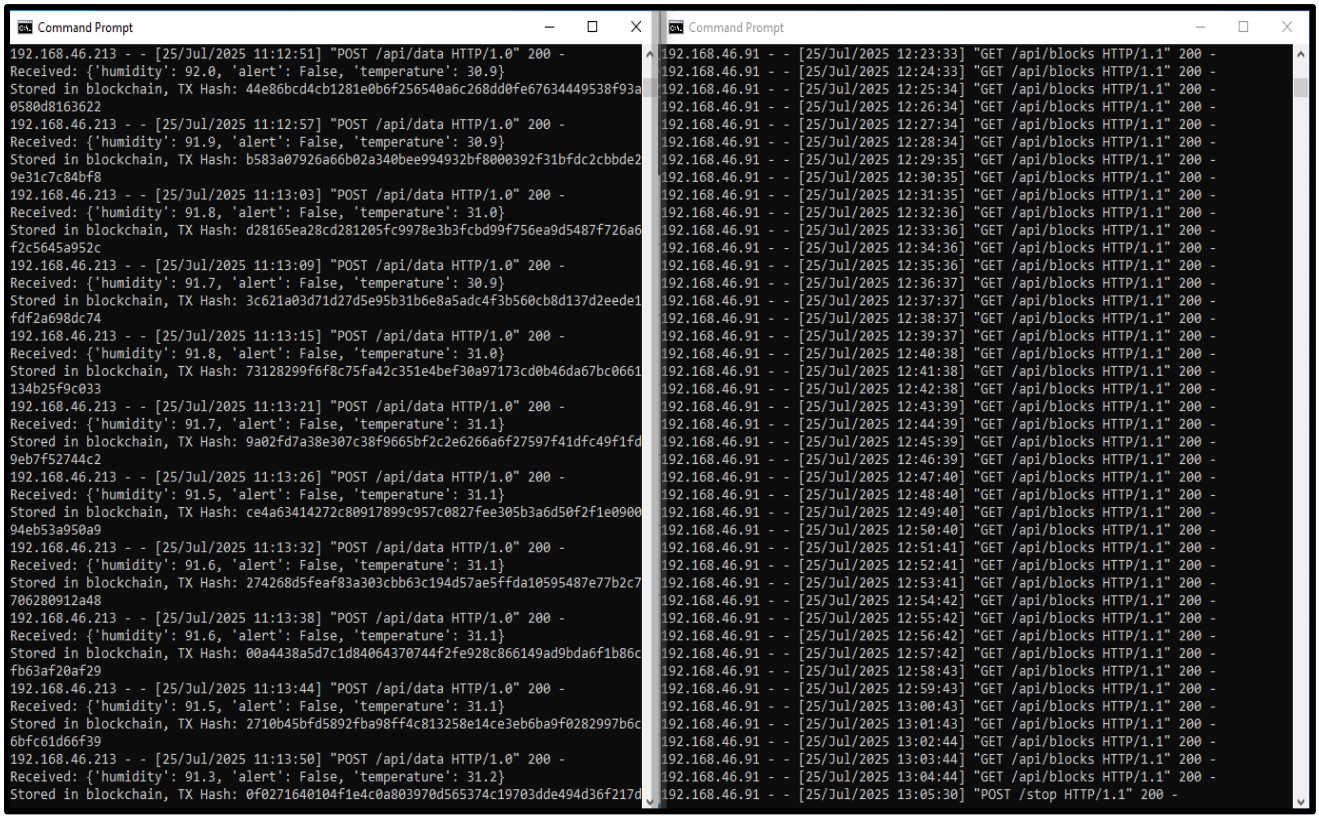
Thonny IDE:


Thonny - MicroPython device :: /main.py @ 61 : 1
File Edit View Run Tools Help
<untitled> [main.py]
sensor = dht.DHT22(machine.Pin(2))
prev_temp = None
prev_hum = None
def percent_change(old, new):
if old is None or old == 0:
return 100
return abs((new - old) / old) * 100
def connect_wifi():
sta = network.WLAN(network.STA_IF)
sta.active(True)
if not sta.isconnected():
print("Connecting to WiFi...")
sta.connect(WIFI_SSID, WIFI_PASS)
Shell
MicroPython v1.25.0 on 2025-04-15; ESP module (1M) with ESP8266
Type "help()" for more information.
>>>
MicroPython (ESP8266) • CP2102 USB to UART Bridge Controller @ COM3

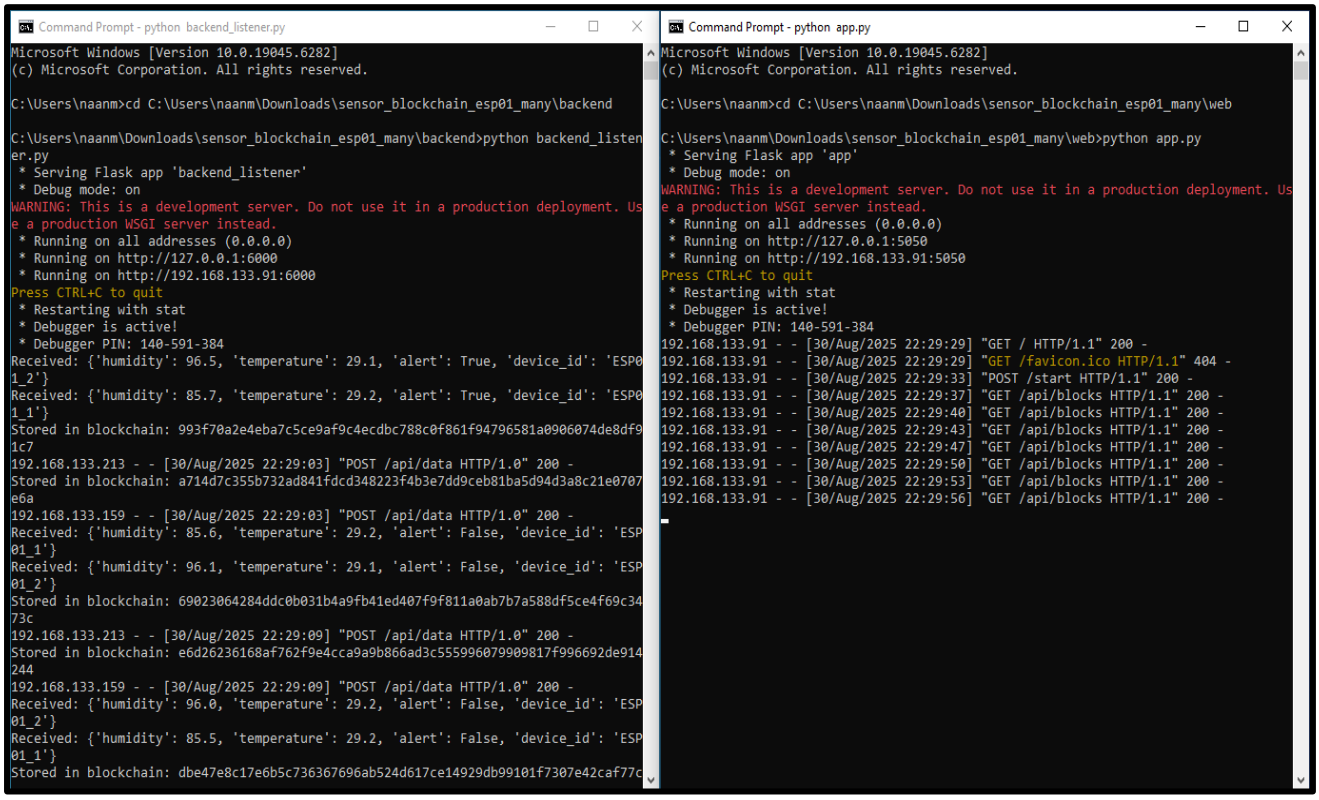


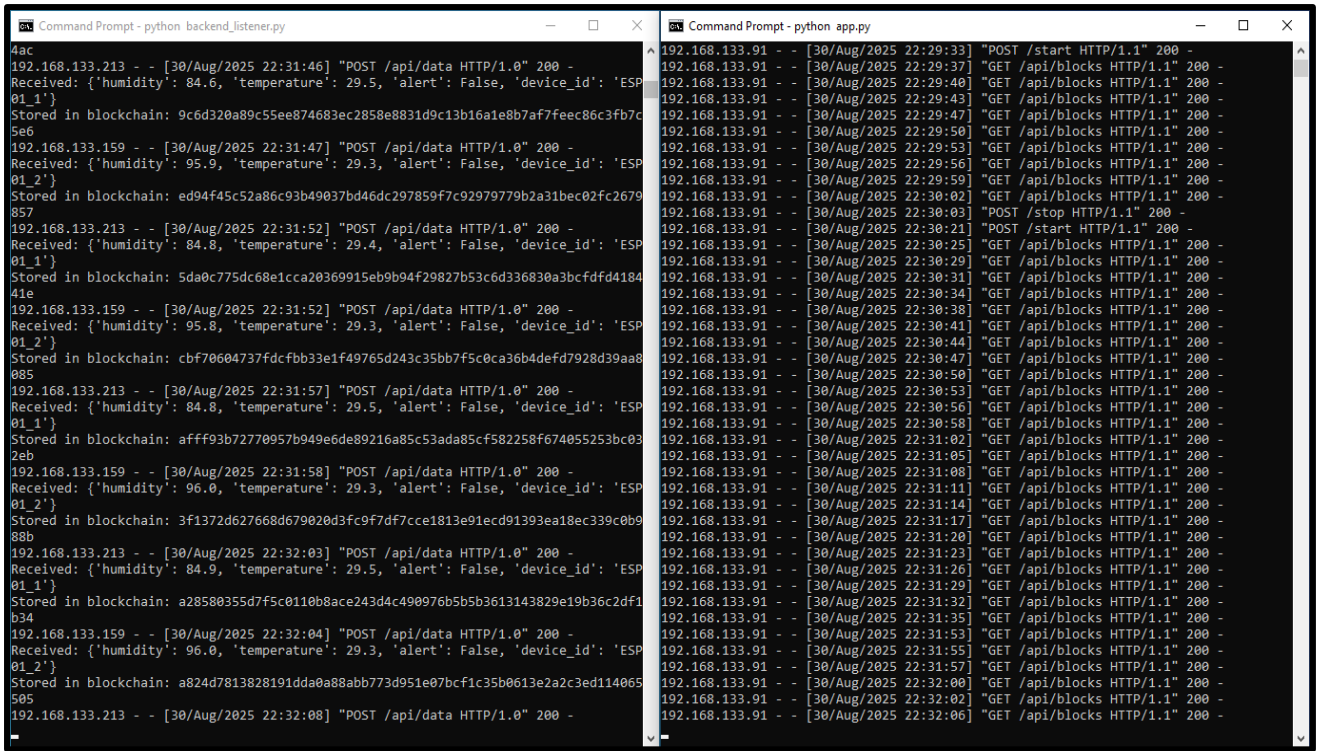
CMD Outputs (Single Device):



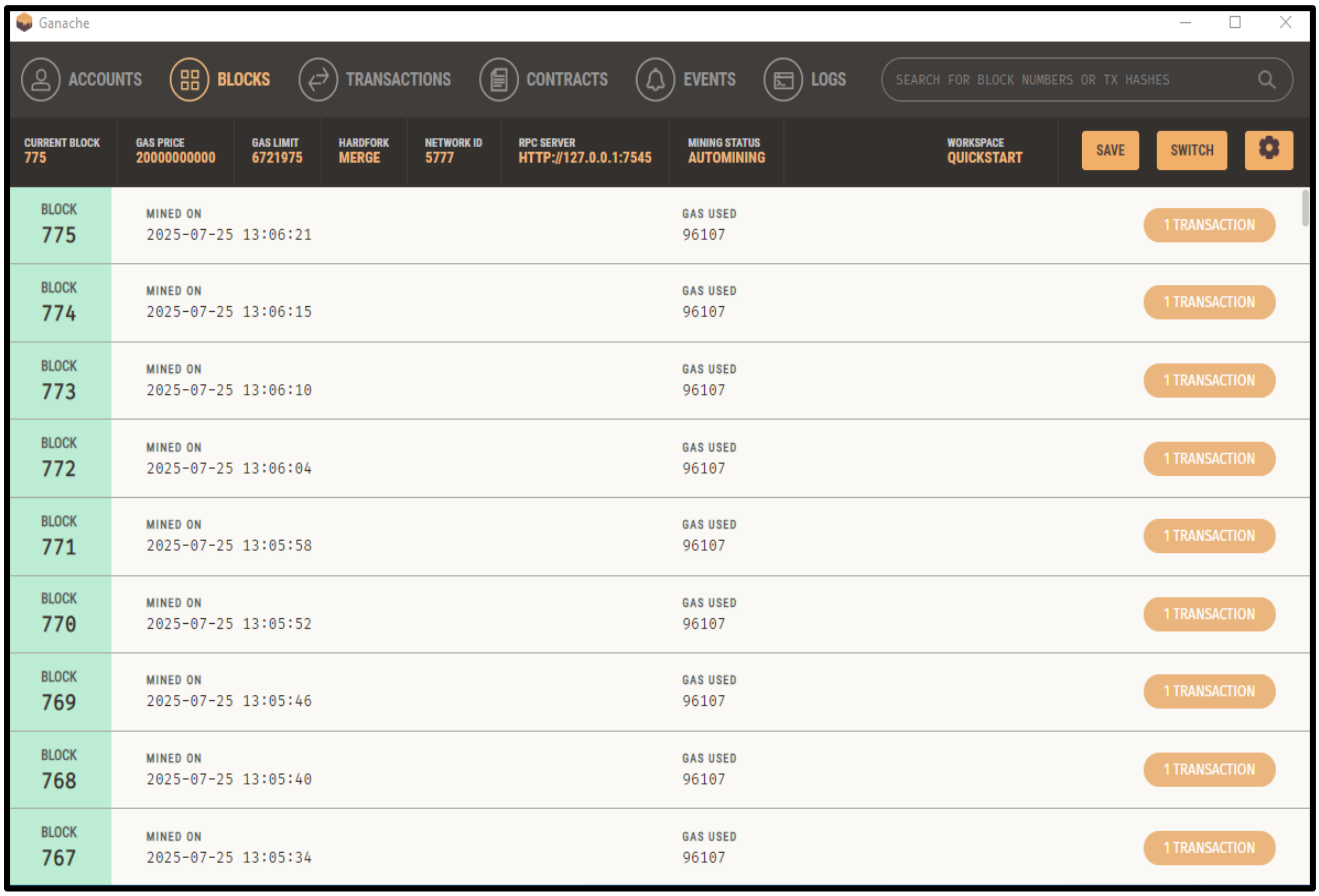


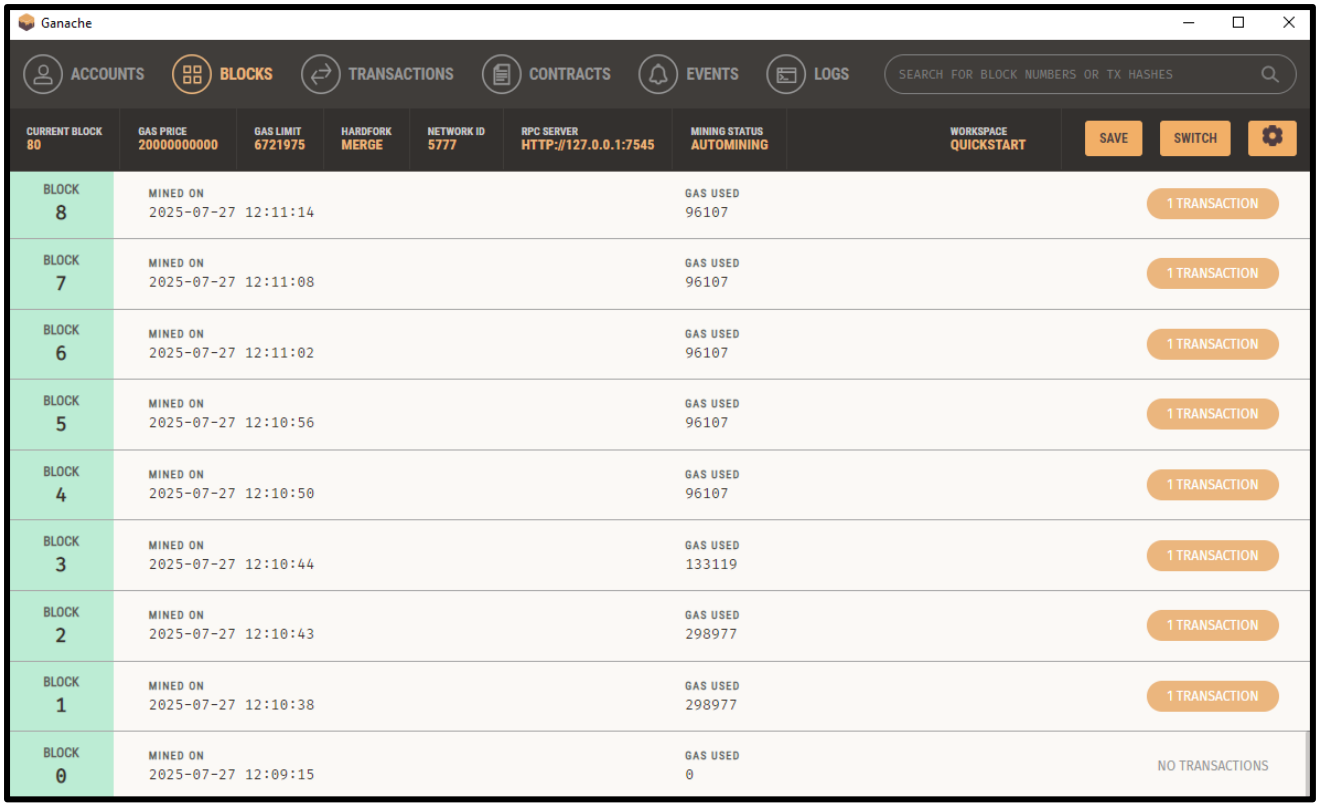
CMD Outputs (Multiple Devices):



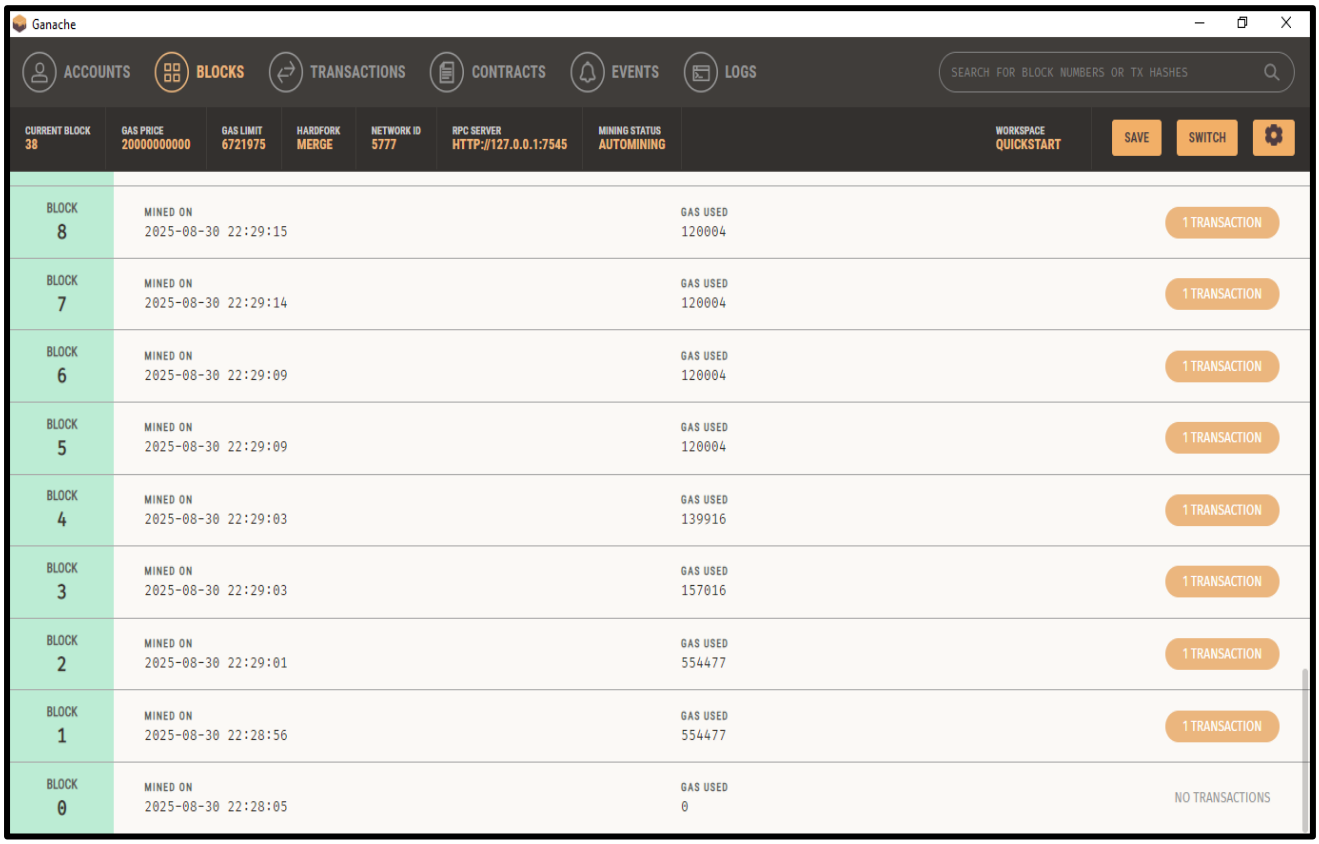


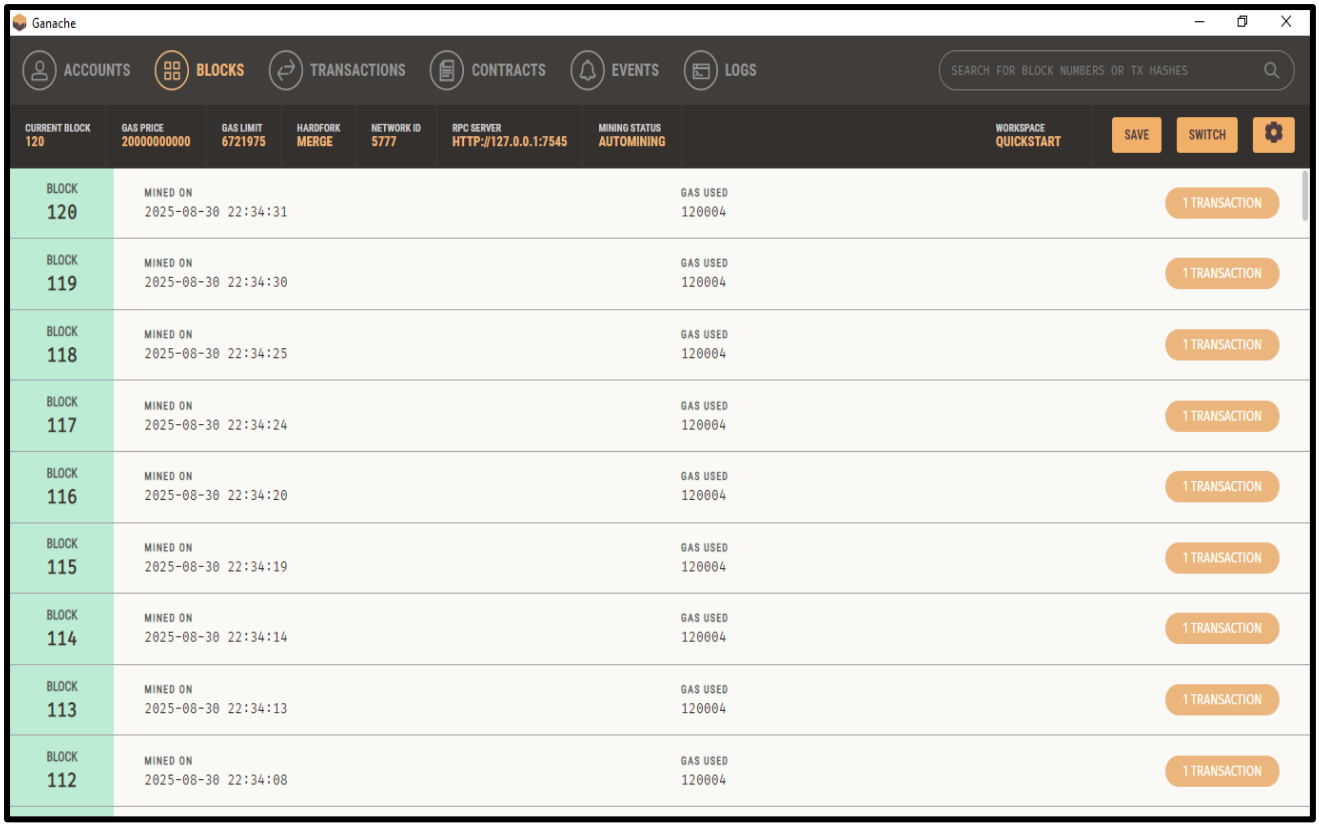
Ganache Blocks (Single Device):



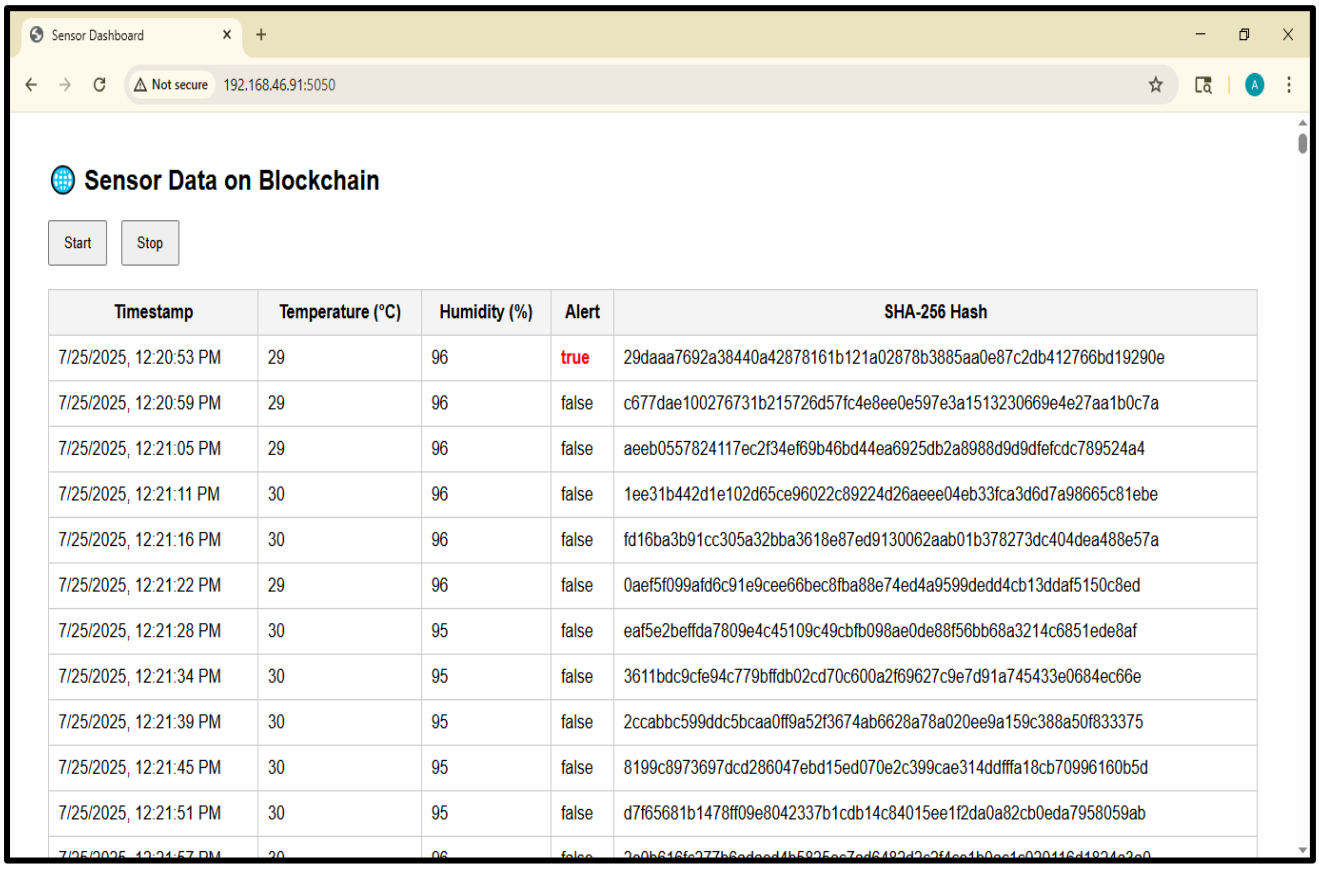


Ganache Blocks (Multiple Devices):

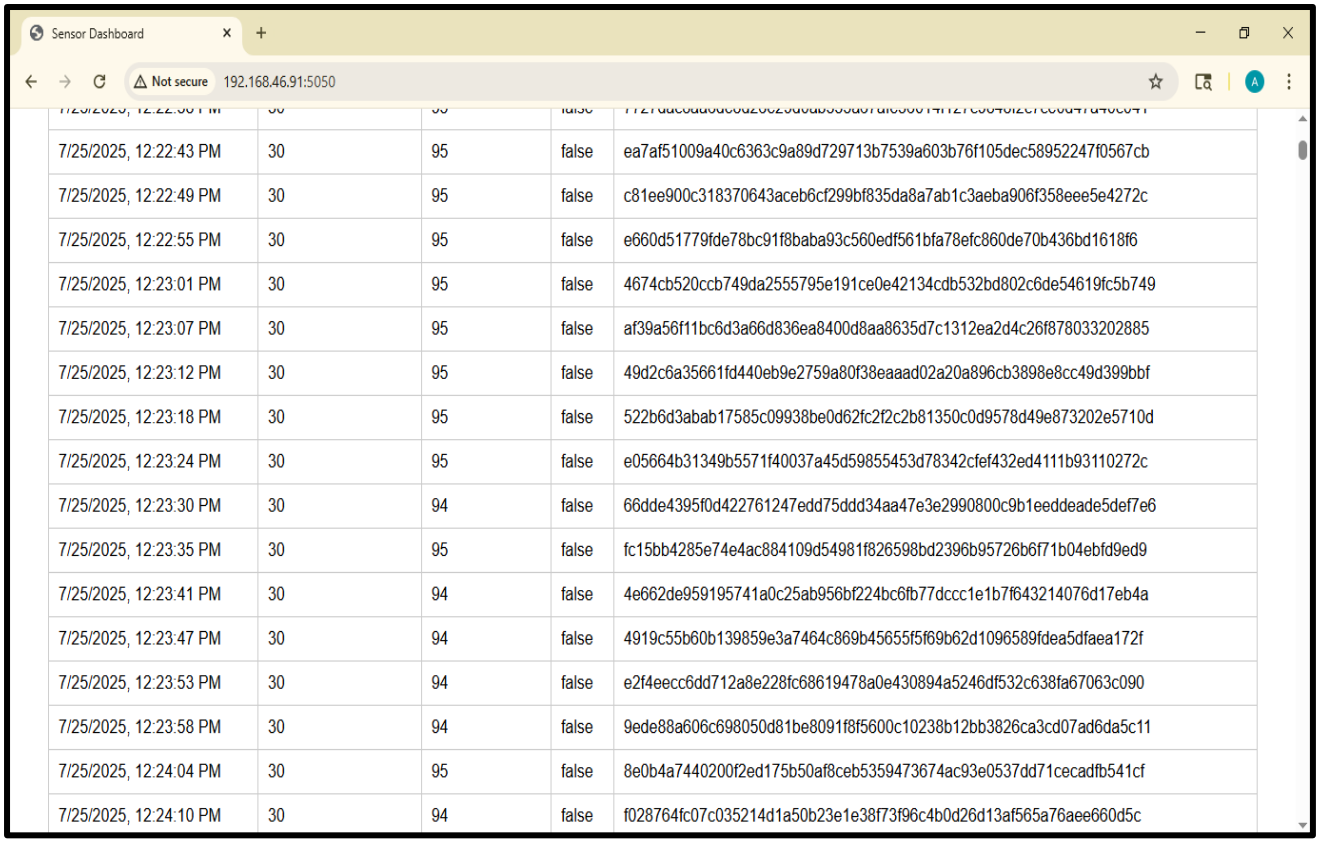




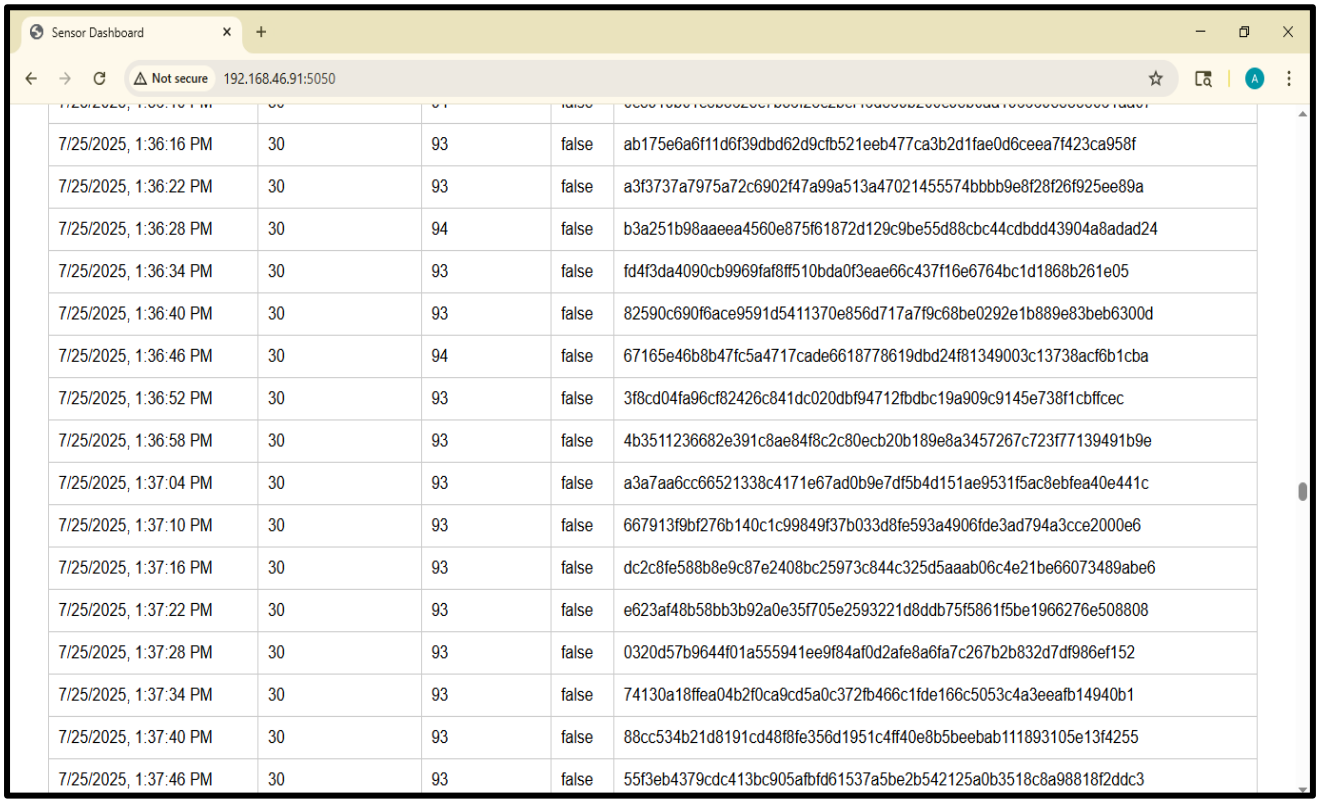
Webpage Readings (Single Device):



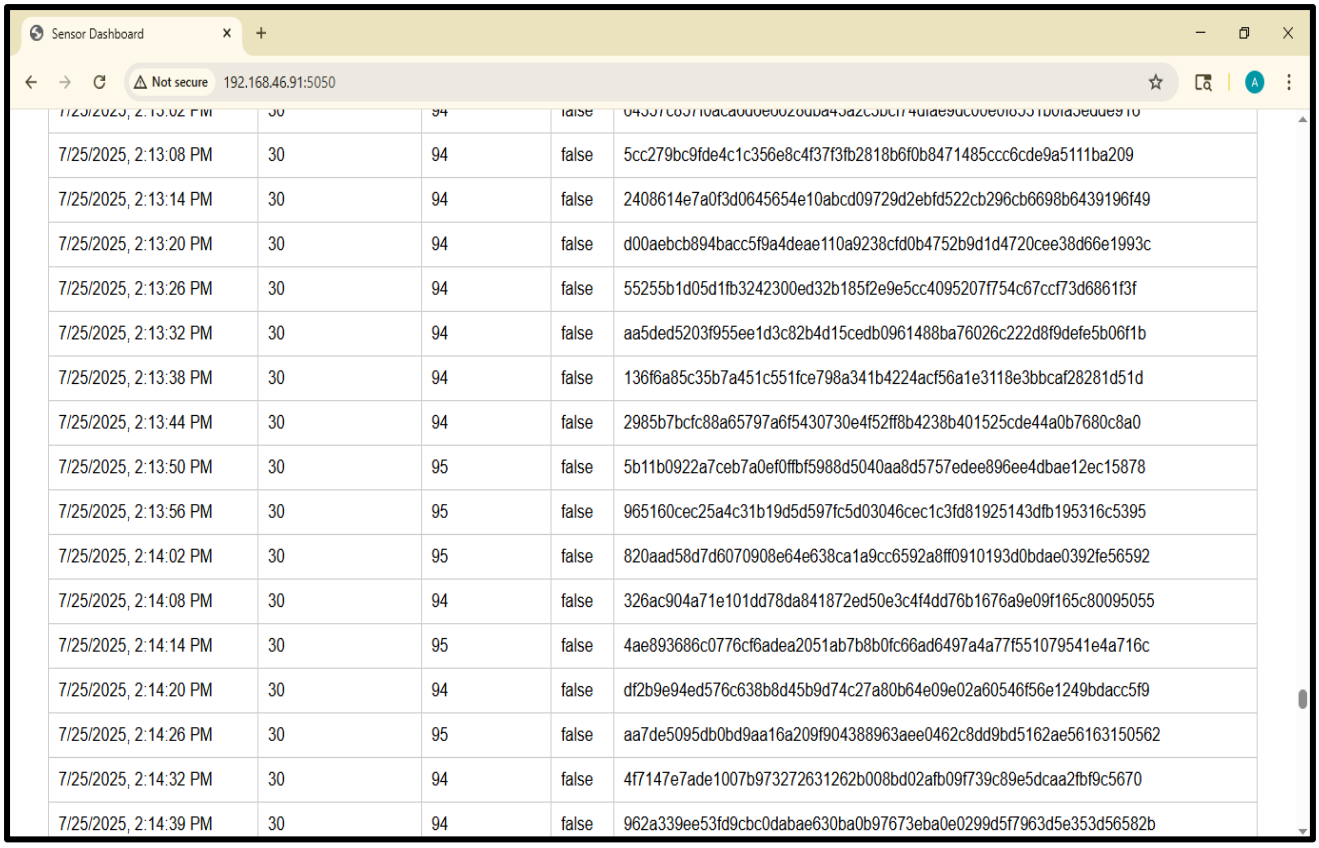
Timestamp	Temperature (°C)	Humidity (%)	Alert	SHA-256 Hash
7/25/2025, 12:20:53 PM	29	96	true	29daaa7692a38440a42878161b121a02878b3885aa0e87c2db412766bd19290e
7/25/2025, 12:20:59 PM	29	96	false	c677dae100276731b215726d57fc4e8ee0e597e3a1513230669e4e27aa1b0c7a
7/25/2025, 12:21:05 PM	29	96	false	aeeb0557824117ec2f34ef69b46bd44ea6925db2a8988d9d9dfefcdc789524a4
7/25/2025, 12:21:11 PM	30	96	false	1ee31b442d1e102d65ce96022c89224d26aeee04eb33fca3d6d7a98665c81ebe
7/25/2025, 12:21:16 PM	30	96	false	fd16ba3b91cc305a32bba3618e87ed9130062aab01b378273dc404dea488e57a
7/25/2025, 12:21:22 PM	29	96	false	0aef5f099afd6c91e9cee66bec8fba88e74ed4a9599dedd4cb13ddaf5150c8ed
7/25/2025, 12:21:28 PM	30	95	false	eaf5e2beffda7809e4c45109c49cbfb098ae0de88f56bb68a3214c6851ede8af
7/25/2025, 12:21:34 PM	30	95	false	3611bdc9cfe94c779bffdb02cd70c600a2f69627c9e7d91a745433e0684ec66e
7/25/2025, 12:21:39 PM	30	95	false	2ccabbc599ddc5bcaa0ff9a52f3674ab6628a78a020ee9a159c388a50f833375
7/25/2025, 12:21:45 PM	30	95	false	8199c8973697dcd286047ebd15ed070e2c399cae314ddfffa18cb70996160b5d
7/25/2025, 12:21:51 PM	30	95	false	d7f65681b1478ff09e8042337b1cdb14c84015ee1f2da0a82cb0eda7958059ab



7/25/2025, 12:22:43 PM	30	95	false	ea7af51009a40c6363c9a89d729713b7539a603b76f105dec58952247f0567cb
7/25/2025, 12:22:49 PM	30	95	false	c81ee900c318370643aceb6cf299bf835da8a7ab1c3aeba906f358eee5e4272c
7/25/2025, 12:22:55 PM	30	95	false	e660d51779fde78bc91f8baba93c560edf561bfa78efc860de70b436bd1618f6
7/25/2025, 12:23:01 PM	30	95	false	4674cb520ccb749da2555795e191ce0e42134cdb532bd802c6de54619fc5b749
7/25/2025, 12:23:07 PM	30	95	false	af39a56f11bc6d3a66d836ea8400d8aa8635d7c1312ea2d4c26f878033202885
7/25/2025, 12:23:12 PM	30	95	false	49d2c6a35661fd440eb9e2759a80f38eaaad02a20a896cb3898e8cc49d399bbf
7/25/2025, 12:23:18 PM	30	95	false	522b6d3abab17585c09938be0d62fc2f2c2b81350c0d9578d49e873202e5710d
7/25/2025, 12:23:24 PM	30	95	false	e05664b31349b5571f40037a45d59855453d78342cfef432ed4111b93110272c
7/25/2025, 12:23:30 PM	30	94	false	66dde4395f0d422761247edd75ddd34aa47e3e2990800c9b1eeddeade5def7e6
7/25/2025, 12:23:35 PM	30	95	false	fc15bb4285e74e4ac884109d54981f826598bd2396b95726b6f71b04ebfd9ed9
7/25/2025, 12:23:41 PM	30	94	false	4e662de959195741a0c25ab956bf224bc6fb77dccc1e1b7f643214076d17eb4a
7/25/2025, 12:23:47 PM	30	94	false	4919c55b60b139859e3a7464c869b45655f5f69b62d1096589fdea5dfaea172f
7/25/2025, 12:23:53 PM	30	94	false	e2f4eecc6dd712a8e228fc68619478a0e430894a5246df532c638fa67063c090
7/25/2025, 12:23:58 PM	30	94	false	9ede88a606c698050d81be8091f8f5600c10238b12bb3826ca3cd07ad6da5c11
7/25/2025, 12:24:04 PM	30	95	false	8e0b4a7440200f2ed175b50af8ceb5359473674ac93e0537dd71cecadfb541cf
7/25/2025, 12:24:10 PM	30	94	false	f028764fc07c035214d1a50b23e1e38f73f96c4b0d26d13af565a76aee660d5c

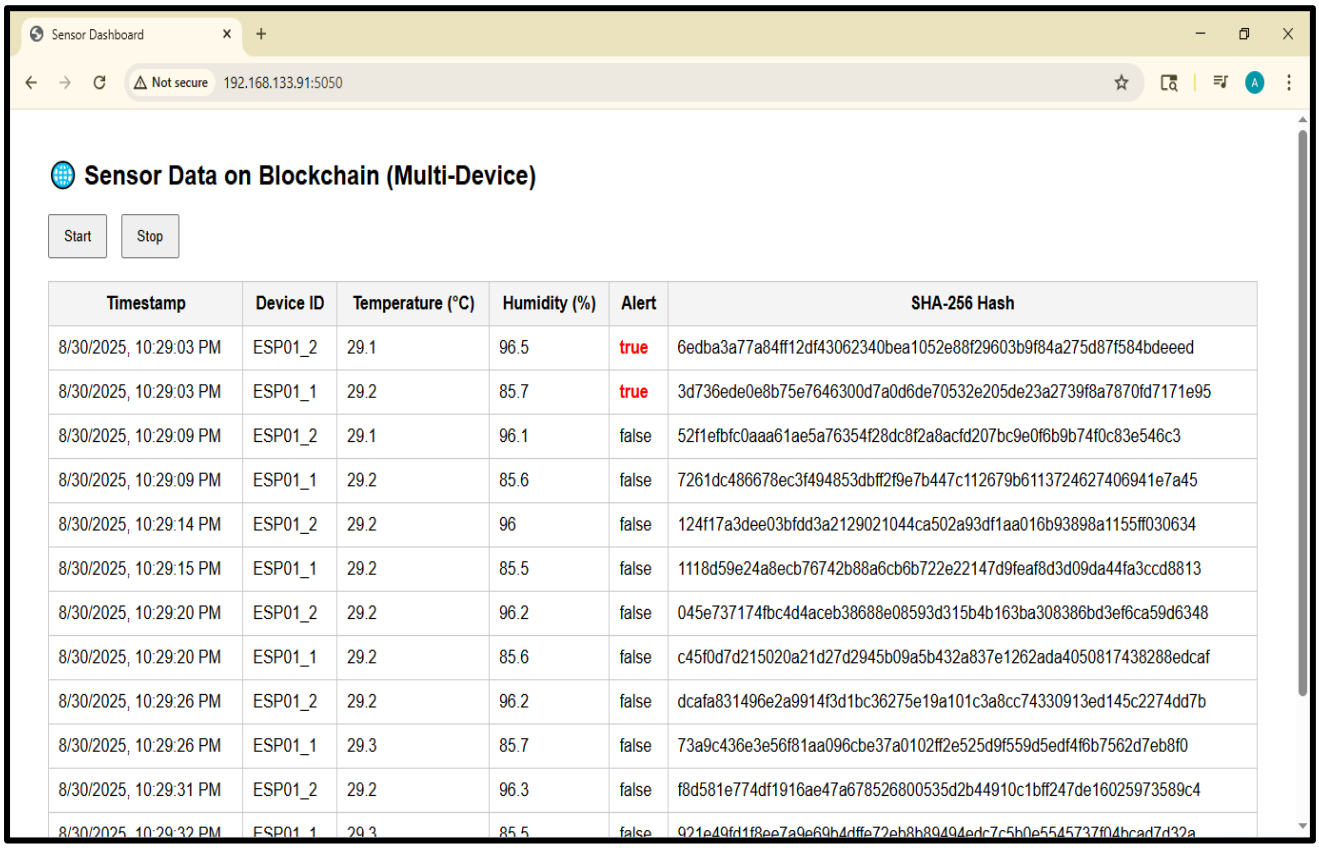


7/25/2025, 1:36:16 PM	30	93	false	ab175e6a6f11d6f39dbd62d9cfb521eeb477ca3b2d1fae0d6ceea7f423ca958f
7/25/2025, 1:36:22 PM	30	93	false	a3f3737a7975a72c6902f47a99a513a47021455574bbbb9e8f28f26f925ee89a
7/25/2025, 1:36:28 PM	30	94	false	b3a251b98aaeea4560e875f61872d129c9be55d88cbc44cdbdd43904a8adad24
7/25/2025, 1:36:34 PM	30	93	false	fd4f3da4090cb9969faf8ff510bda0f3eae66c437f16e6764bc1d1868b261e05
7/25/2025, 1:36:40 PM	30	93	false	82590c690f6ace9591d5411370e856d717a7f9c68be0292e1b889e83beb6300d
7/25/2025, 1:36:46 PM	30	94	false	67165e46b8b47fc5a4717cade6618778619dbd24f81349003c13738acf6b1cba
7/25/2025, 1:36:52 PM	30	93	false	3f8cd04fa96cf82426c841dc020dbf94712fbdbc19a909c9145e738f1cbffcec
7/25/2025, 1:36:58 PM	30	93	false	4b3511236682e391c8ae84f8c2c80ecb20b189e8a3457267c723f77139491b9e
7/25/2025, 1:37:04 PM	30	93	false	a3a7aa6cc66521338c4171e67ad0b9e7df5b4d151ae9531f5ac8ebfea40e441c
7/25/2025, 1:37:10 PM	30	93	false	667913f9bf276b140c1c99849f37b033d8fe593a4906fde3ad794a3cce2000e6
7/25/2025, 1:37:16 PM	30	93	false	dc2c8fe588b8e9c87e2408bc25973c844c325d5aaab06c4e21be66073489abe6
7/25/2025, 1:37:22 PM	30	93	false	e623af48b58bb3b92a0e35f705e2593221d8ddb75f5861f5be1966276e508808
7/25/2025, 1:37:28 PM	30	93	false	0320d57b9644f01a555941ee9f84af0d2afe8a6fa7c267b2b832d7df986ef152
7/25/2025, 1:37:34 PM	30	93	false	74130a18ffea04b2f0ca9cd5a0c372fb466c1fde166c5053c4a3eeafb14940b1
7/25/2025, 1:37:40 PM	30	93	false	88cc534b21d8191cd48f8fe356d1951c4ff40e8b5beebab111893105e13f4255
7/25/2025, 1:37:46 PM	30	93	false	55f3eb4379cdc413bc905afbfd61537a5be2b542125a0b3518c8a98818f2ddc3

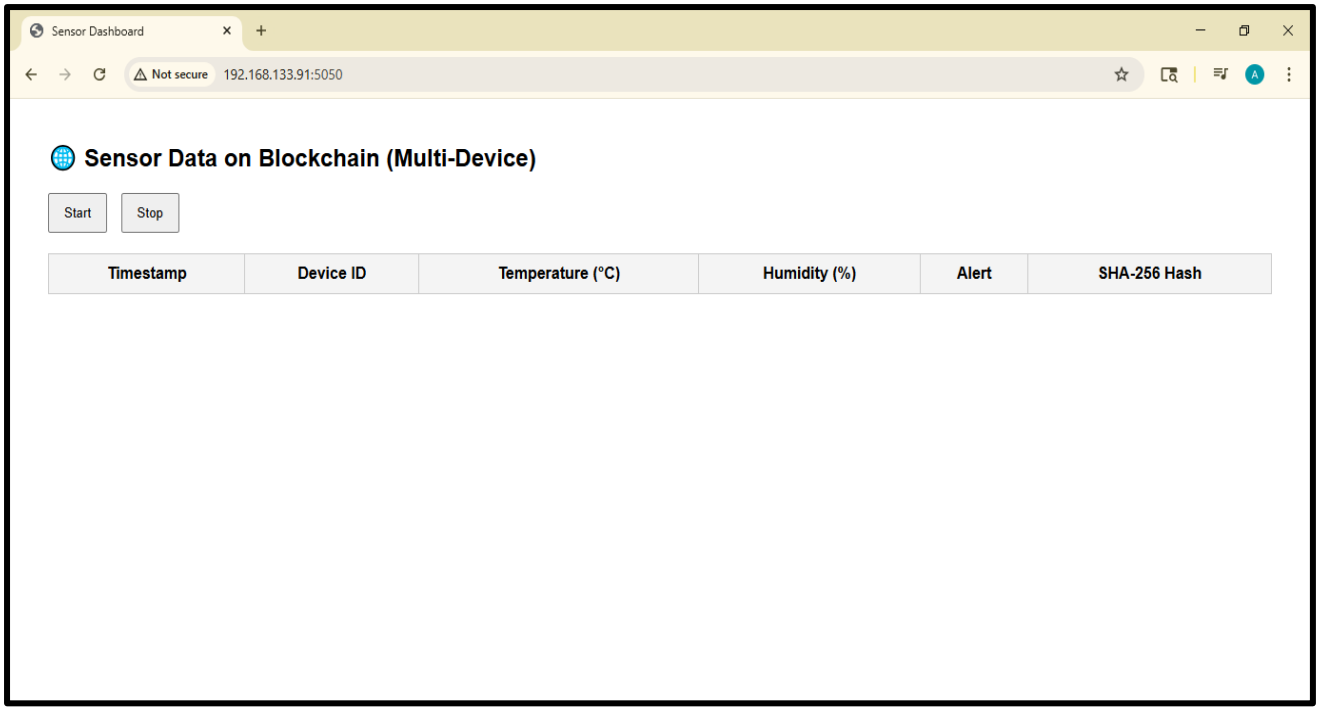


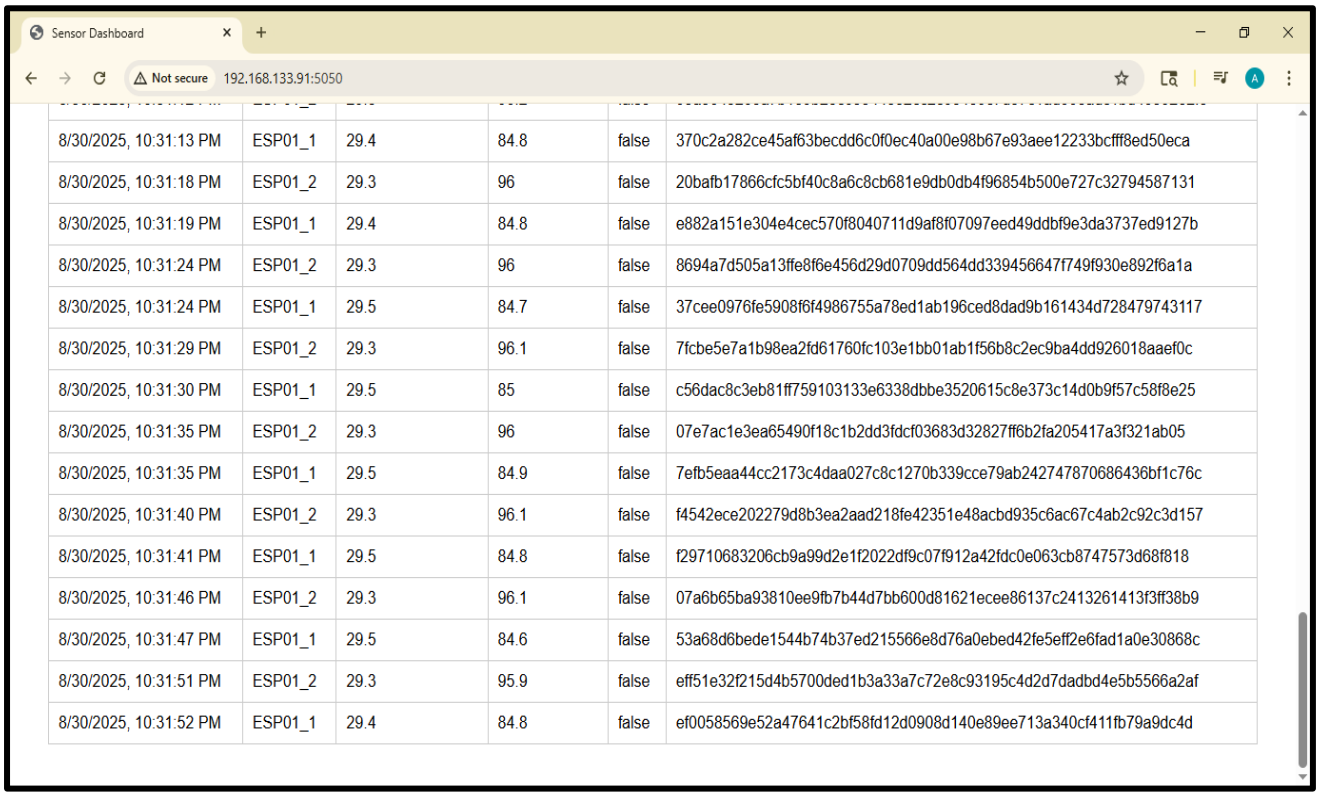
7/25/2025, 2:13:08 PM	30	94	false	5cc279bc9fde4c1c356e8c4f37f3fb2818b6f0b8471485ccc6cde9a5111ba209
7/25/2025, 2:13:14 PM	30	94	false	2408614e7a0f3d0645654e10abcd09729d2ebfd522cb296cb6698b6439196f49
7/25/2025, 2:13:20 PM	30	94	false	d00aebcb894bacc5f9a4deae110a9238cfd0b4752b9d1d4720cee38d66e1993c
7/25/2025, 2:13:26 PM	30	94	false	55255b1d05d1fb3242300ed32b185f2e9e5cc4095207f754c67ccf73d6861f3f
7/25/2025, 2:13:32 PM	30	94	false	aa5ded5203f955ee1d3c82b4d15cedb0961488ba76026c222d8f9defe5b06f1b
7/25/2025, 2:13:38 PM	30	94	false	136f6a85c35b7a451c551fce798a341b4224acf56a1e3118e3bbcaf28281d51d
7/25/2025, 2:13:44 PM	30	94	false	2985b7bcfc88a65797a6f5430730e4f52ff8b4238b401525cde44a0b7680c8a0
7/25/2025, 2:13:50 PM	30	95	false	5b11b0922a7ceb7a0ef0ffbf5988d5040aa8d5757edee896ee4dbae12ec15878
7/25/2025, 2:13:56 PM	30	95	false	965160cec25a4c31b19d5d597fc5d03046cec1c3fd81925143dfb195316c5395
7/25/2025, 2:14:02 PM	30	95	false	820aad58d7d6070908e64e638ca1a9cc6592a8ff0910193d0bdae0392fe56592
7/25/2025, 2:14:08 PM	30	94	false	326ac904a71e101dd78da841872ed50e3c4f4dd76b1676a9e09f165c80095055
7/25/2025, 2:14:14 PM	30	95	false	4ae893686c0776cf6adea2051ab7b8b0fc66ad6497a4a77f551079541e4a716c
7/25/2025, 2:14:20 PM	30	94	false	df2b9e94ed576c638b8d45b9d74c27a80b64e09e02a60546f56e1249bdacc5f9
7/25/2025, 2:14:26 PM	30	95	false	aa7de5095db0bd9aa16a209f904388963aee0462c8dd9bd5162ae56163150562
7/25/2025, 2:14:32 PM	30	94	false	4f7147e7ade1007b973272631262b008bd02afb09f739c89e5dcaa2fbf9c5670
7/25/2025, 2:14:39 PM	30	94	false	962a339ee53fd9cbc0dabae630ba0b97673eba0e0299d5f7963d5e353d56582b

Webpage Readings (Multiple Devices):



Timestamp	Device ID	Temperature (°C)	Humidity (%)	Alert	SHA-256 Hash
8/30/2025, 10:29:03 PM	ESP01_2	29.1	96.5	true	6edba3a77a84ff12df43062340bea1052e88f29603b9f84a275d87f584bdeeed
8/30/2025, 10:29:03 PM	ESP01_1	29.2	85.7	true	3d736ede0e8b75e7646300d7a0d6de70532e205de23a2739f8a7870fd7171e95
8/30/2025, 10:29:09 PM	ESP01_2	29.1	96.1	false	52f1efbfc0aaa61ae5a76354f28dc8f2a8acfd207bc9e0f6b9b74f0c83e546c3
8/30/2025, 10:29:09 PM	ESP01_1	29.2	85.6	false	7261dc486678ec3f494853dbff2f9e7b447c112679b61137246274069 41e7a45
8/30/2025, 10:29:14 PM	ESP01_2	29.2	96	false	124f17a3dee03bfdd3a2129021044ca502a93df1aa016b93898a1155ff030634
8/30/2025, 10:29:15 PM	ESP01_1	29.2	85.5	false	1118d59e24a8ecb76742b88a6cb6b722e22147d9feaf8d3d09da44fa3ccd8813
8/30/2025, 10:29:20 PM	ESP01_2	29.2	96.2	false	045e737174fbc4d4aceb38688e08593d315b4b163ba308386bd3ef6ca59d6348
8/30/2025, 10:29:20 PM	ESP01_1	29.2	85.6	false	c45f0d7d215020a21d27d2945b09a5b432a837e1262ada4050817438288edcaf
8/30/2025, 10:29:26 PM	ESP01_2	29.2	96.2	false	dcafa831496e2a9914f3d1bc36275e19a101c3a8cc74330913ed145c2274dd7b
8/30/2025, 10:29:26 PM	ESP01_1	29.3	85.7	false	73a9c436e3e56f81aa096cbe37a0102ff2e525d9f559d5edf4f6b7562d7eb8f0
8/30/2025, 10:29:31 PM	ESP01_2	29.2	96.3	false	f8d581e774df1916ae47a678526800535d2b44910c1bff247de16025973589c4
8/30/2025, 10:29:32 PM	ESP01_1	29.3	85.5	false	921e49fd1f8ee7a9e69b4dffe72eb8b89494edc7c5b0e5545737f04bcad7d32a





8/30/2025, 10:31:13 PM	ESP01_1	29.4	84.8	false	370c2a282ce45af63becdd6c0f0ec40a00e98b67e93aee12233bcfff8ed50eca
8/30/2025, 10:31:18 PM	ESP01_2	29.3	96	false	20bafb17866cfc5bf40c8a6c8cb681e9db0db4f96854b500e727c32794587131
8/30/2025, 10:31:19 PM	ESP01_1	29.4	84.8	false	e882a151e304e4cec570f8040711d9af8f07097eed49ddbf9e3da3737ed9127b
8/30/2025, 10:31:24 PM	ESP01_2	29.3	96	false	8694a7d505a13ffe8f6e456d29d0709dd564dd339456647f749f930e892f6a1a
8/30/2025, 10:31:24 PM	ESP01_1	29.5	84.7	false	37cee0976fe5908f6f4986755a78ed1ab196ced8dad9b161434d728479743117
8/30/2025, 10:31:29 PM	ESP01_2	29.3	96.1	false	7fcbe5e7a1b98ea2fd61760fc103e1bb01ab1f56b8c2ec9ba4dd926018aaef0c
8/30/2025, 10:31:30 PM	ESP01_1	29.5	85	false	c56dac8c3eb81ff759103133e6338dbbe3520615c8e373c14d0b9f57c58f8e25
8/30/2025, 10:31:35 PM	ESP01_2	29.3	96	false	07e7ac1e3ea65490f18c1b2dd3fdcf03683d32827ff6b2fa205417a3f321ab05
8/30/2025, 10:31:35 PM	ESP01_1	29.5	84.9	false	7efb5eaa44cc2173c4daa027c8c1270b339cce79ab242747870686436bf1c76c
8/30/2025, 10:31:40 PM	ESP01_2	29.3	96.1	false	f4542ece202279d8b3ea2aad218fe42351e48acbd935c6ac67c4ab2c92c3d157
8/30/2025, 10:31:41 PM	ESP01_1	29.5	84.8	false	f29710683206cb9a99d2e1f2022df9c07f912a42fdc0e063cb8747573d68f818
8/30/2025, 10:31:46 PM	ESP01_2	29.3	96.1	false	07a6b65ba93810ee9fb7b44d7bb600d81621ecee86137c2413261413f3ff38b9
8/30/2025, 10:31:47 PM	ESP01_1	29.5	84.6	false	53a68d6bede1544b74b37ed215566e8d76a0ebed42fe5eff2e6fad1a0e30868c
8/30/2025, 10:31:51 PM	ESP01_2	29.3	95.9	false	eff51e32f215d4b5700ded1b3a33a7c72e8c93195c4d2d7dadbd4e5b5566a2af
8/30/2025, 10:31:52 PM	ESP01_1	29.4	84.8	false	ef0058569e52a47641c2bf58fd12d0908d140e89ee713a340cf411fb79a9dc4d

7.2 Test Cases:

Test Case No.	Test Case Scenario	Operator action	Expected Result	Actual Result	Test Result
Hardware					
TC01	Start the project	Connect battery	Project should power up	Project powers up	Successful
TC02	Connecting Wi-Fi	Check client & server network	ESP-01 & server should be connected	ESP-01& server are connected	Successful
Software					
TC03	Open Ganache	Run Ganache	Ethereum chain to be initialized	Ethereum chain is initialized	Successful
TC04	Start backend listener	Run backend listener code	Sensor data packets to be received on terminal	Sensor data packets are received on terminal	Successful
TC05	Check Ganache	Open Ganache network	Blocks should be mining	Blocks are mining	Successful
Web Page					
TC06	Start web app	Run web app code	Web server should start	Web server is started	Successful
TC07	Check web page	Open a browser and enter the web address	Web page should load	Web page loads	Successful
TC08	Start web page readings	Click start on webpage	Sensor data readings should be displayed	Sensor data readings are displayed	Successful

TC09	Stop web page readings	Click stop on webpage	Sensor data readings should not be displayed	Sensor data readings stop displaying	Successful
TC10	Check Ganache	Open Ganache network	Blocks to be mined	Blocks are mined	Successful
Performance					
TC11	Mining multiple blocks	Connect two or more clients	Every client block should be mined	Every client block is mined	Successful
TC12	Check temperature accuracy	Check temperature	Temperature should match	Temperature matches	Successful
TC13	Check humidity accuracy	Check humidity	Humidity should match	Humidity matches	Successful

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion:

The developed system demonstrates how the integration of IoT devices with blockchain technology can address the challenges of data security, authenticity, and transparency in temperature and humidity monitoring. By utilizing low-cost IoT hardware for environmental sensing and a decentralized blockchain ledger for secure data recording, the system ensures that sensor data remains tamper-proof, verifiable, and permanently traceable. This combination overcomes the limitations of conventional centralized monitoring systems, which are often vulnerable to manipulation, single-point failures, and restricted accessibility. Furthermore, the system enables real-time monitoring and automated recording, making it highly suitable for environments where accurate and reliable climate data is critical.

Beyond its technical demonstration, the project establishes a scalable and adaptable framework that can be extended to multiple domains, including agriculture for crop monitoring, supply chain logistics for perishable goods, healthcare for climate-sensitive equipment, and smart cities for environmental regulation. By coupling IoT's sensing and connectivity capabilities with blockchain's immutable and distributed architecture, the system contributes to building trustworthy digital ecosystems. Its practical usability, coupled with the assurance of data integrity, positions this work as a promising step toward the next generation of secure IoT-driven applications in real-world scenarios.

8.2 Future Scope:

1. Extend the system (processing power and storage) to support large-scale IoT deployments with hundreds of sensor nodes.

2. Integration of more diverse sensors (air quality, CO_2, soil moisture, etc.) to expand environmental monitoring capabilities.

3. Deployment on a public blockchain or hybrid blockchain model for wider accessibility and stronger decentralization.

4. Enabling cross-platform communication between different IoT networks and blockchain systems for interoperability.

5. Exploring renewable energy-powered IoT nodes for sustainable system operations for Green IoT Deployment.

9. REFERENCES

9.1 References:

1) Journal and Conference Papers:

[1] Ali, M., Iqbal, M., Riaz, M. F., Usman, M., & Mahmood, A. (2024). Blockchain-enabled sustainable IoT: Challenges, solutions, and future research directions. Sustainability, 16(23), 10177. https://doi.org/10.3390/su162310177

[2] Xu, J., Wei, X., Guo, T., & Yu, Z. (2019). A survey of blockchain applications in different domains. arXiv preprint arXiv:1906.00245. https://arxiv.org/abs/1906.00245

[3] Reyna, A., Martín, C., Chen, J., Soler, E., & Díaz, M. (2018). On blockchain and its integration with IoT: Challenges and opportunities. Computer Communications, 136, 10–29. https://doi.org/10.1016/j.comcom.2018.01.016

[4] Huang, Q., Yang, H., Xu, L., & Zhang, Y. (2025). Low-power IoT sensing with blockchain-enabled secure communication. Sensors, 25(9), 2886. https://doi.org/10.3390/s25092886

2) Technical Documentation:

[5] MicroPython. (n.d.). ESP8266 tutorial index. MicroPython Documentation. https://docs.micropython.org/en/latest/esp8266/tutorial/index.html

[6] MicroPython. (n.d.). ESP8266 tutorial introduction. MicroPython Documentation. https://docs.micropython.org/en/latest/esp8266/tutorial/intro.html

[7] Aosong Electronics. (2011). AM2302/DHT22 temperature and humidity sensor datasheet. https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf

3) Tutorials:

[8] Santos, R. (2020). MicroPython ESP32/ESP8266 DHT11/DHT22 web server. Random Nerd Tutorials. https://randomnerdtutorials.com/micropython-esp32-esp8266-dht11-dht22-web-server/

[9] Grinberg, M. (2018). MicroPython and the Internet of Things, Part V: Temperature and humidity. https://blog.miguelgrinberg.com/post/micropython-and-the-internet-of-things-part-v-temperature-and-humidity

[10] Santos, R. (2019). ESP32/ESP8266 DHT11/DHT22 with MicroPython. Random Nerd Tutorials. https://randomnerdtutorials.com/esp32-esp8266-dht11-dht22-micropython-temperature-humidity-sensor/

4) API Documentation:

[11] PySerial. (n.d.). PySerial API documentation. https://pyserial.readthedocs.io/en/latest/pyserial_api.html

5) Blogs:

[12] Stack Overflow. (2009). How big can a memory-mapped file be?. https://stackoverflow.com/questions/726471/how-big-can-a-memory-mapped-file-be

[13] .NET Runtime Team. (2024). Memory-mapped file issue #97316. GitHub. https://github.com/dotnet/runtime/issues/97316

[14] DigitalOcean. (2017). How to make a web application using Flask in Python 3. DigitalOcean Tutorials. https://www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3

[15] InfluxData. (2023). Building an IoT app with InfluxDB Cloud, Python, and Flask (Part 3). InfluxData Blog. https://www.influxdata.com/blog/building-an-iot-app-with-influxdb-cloud-python-and-flask-part-3/

6) Survey Paper:

[16] Bhattacharya, S., Banerjee, S., Roy, S., & Ghosh, A. (2023). Blockchain-enabled IoT systems: A survey on security, privacy, and architecture. Computer Networks, 237, 110000. https://doi.org/10.1016/j.comnet.2023.110000

7) Books:

[17] Tollervey, N. H. (2017). Programming with MicroPython: Embedded programming with microcontrollers and Python. O'Reilly Media.

[18] Bashir, I. (2020). Mastering blockchain: Unlocking the potential of distributed ledger technology (3rd ed.). Packt Publishing.

[19] Schwartz, M. (2016). Internet of Things with ESP8266. Packt Publishing.

8) AI Source:

[20] OpenAI. (2025). ChatGPT (August 29 version) [Large language model]. https://chat.openai.com/